

JC4001: Distributed Systems

Topic 2: Architectures

Xiaonan Liu

xiaonan.liu@abdn.ac.uk



Content

- Architectural Styles
 - Layered architecture
 - Service-oriented architecture
 - Publish-subscribe architecture
- Middleware
- Layered-system Architectures
 - Simple client-server architecture
 - Multitiered architecture
- Symmetrically distributed system architectures
 - Structured P2P systems
 - Unstructured P2P systems
 - Hierarchically organized P2P networks
- Hybrid system architectures
 - Cloud computing
 - Edge-cloud architecture
 - Blockchain architecture

Architectural Styles

Basic idea

A style is formulated in terms of

- (replaceable) components with well-defined interfaces
- the way that components are connected to each other
- the data exchanged between components
- how these components and connectors are jointly configured into a system.

A component is a modular unit with clearly defined interfaces, both required and provided, and is designed to be replaceable within its environment. The ability to replace a component without disrupting the system's operation is crucial. Often, it's not feasible to shut down a system for maintenance; instead, parts of it may be temporarily taken offline. Replacement of a component is feasible only if its interfaces remain unchanged. In practice, this often means that a server or part of a system undergoes regular updates and switches to the updated components once the installation is complete. If a system part must be restarted for updates to take effect, special measures can ensure continuity during the restart process.

Architectural Styles

Connector

A mechanism that mediates communication, coordination, or cooperation among components. **Example:** facilities for (remote) procedure call, messaging, or streaming. Connectors enable the flow of control and data between components.

With components and connectors, we can create various configurations classified into architectural styles.

Key Styles

Layered architectures

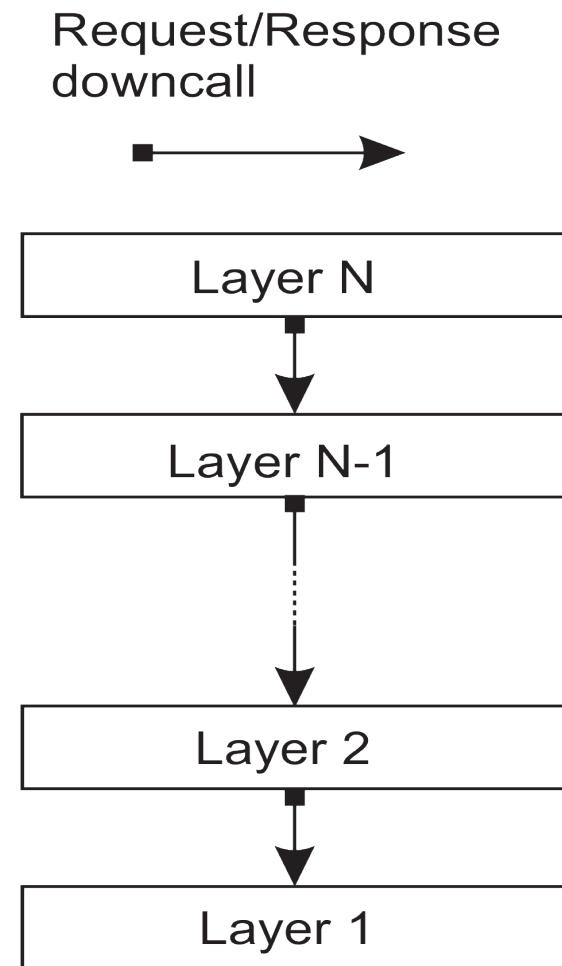
Service-oriented architectures

Publish-subscribe architectures

Layered Architecture

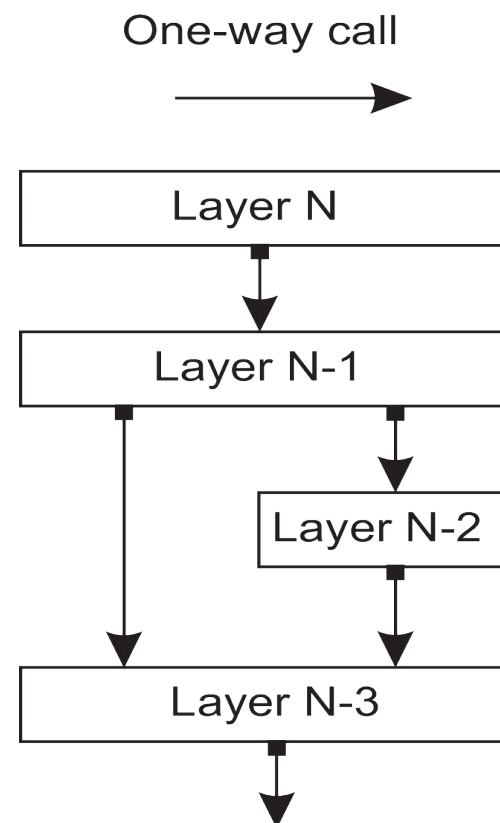
Different layered organizations

The basic idea for the layered style is simple: components are organized in a layered fashion where a component at layer L_j can make a downcall to a component at a lower-level layer L_i (with $i < j$) and generally expects a response. Only in exceptional cases will an upcall be made to a higher-level component.



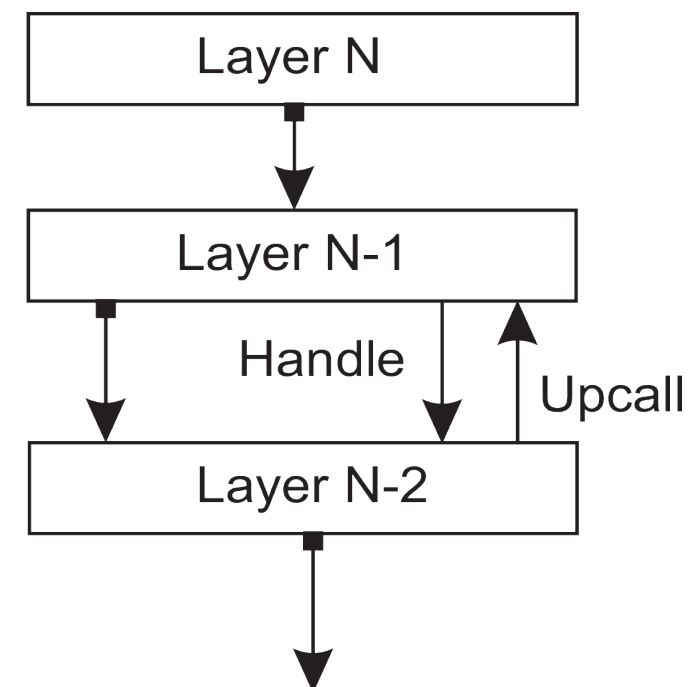
(a)

Pure layered organization



(b)

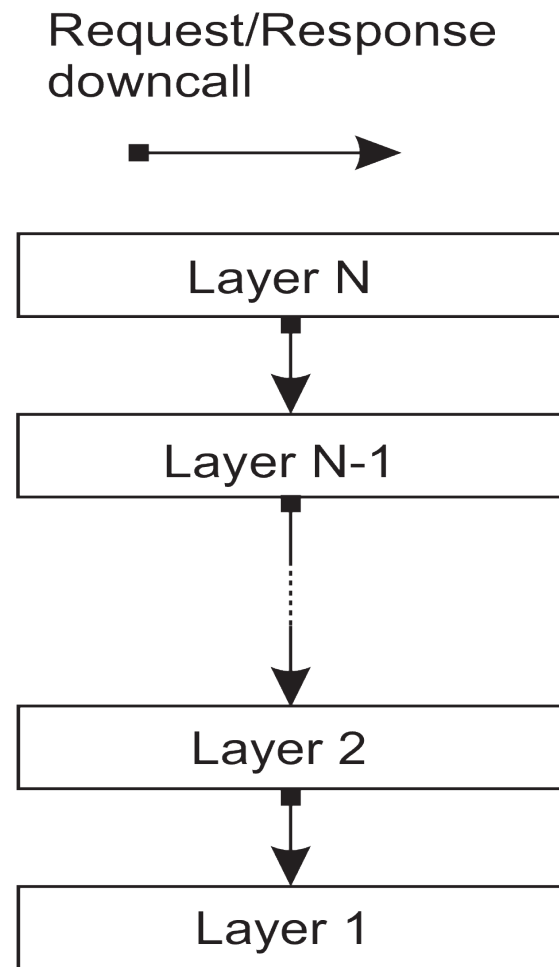
Mixed layered organization



(c)

Layered organization with upcalls

Layered Architecture

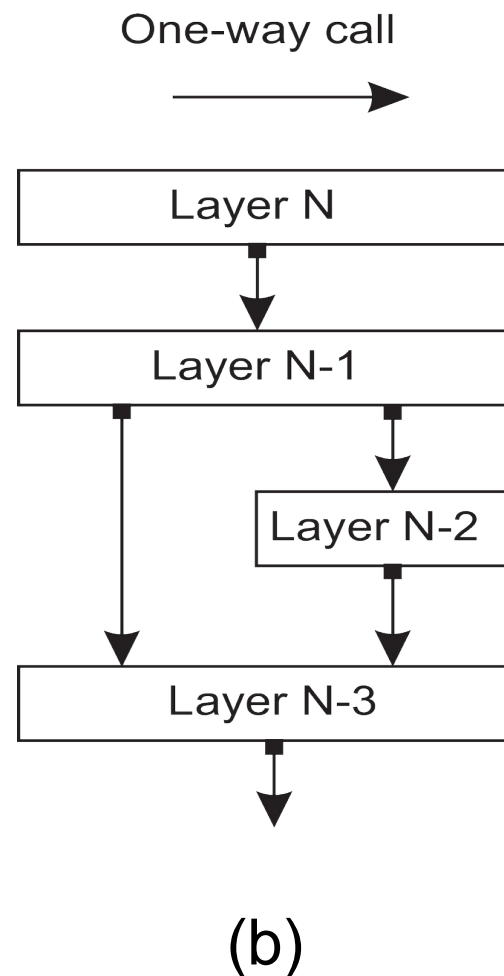


(a)

Pure layered organization

It shows a standard organization in which only downcalls to the next lower layer are made.

Layered Architecture

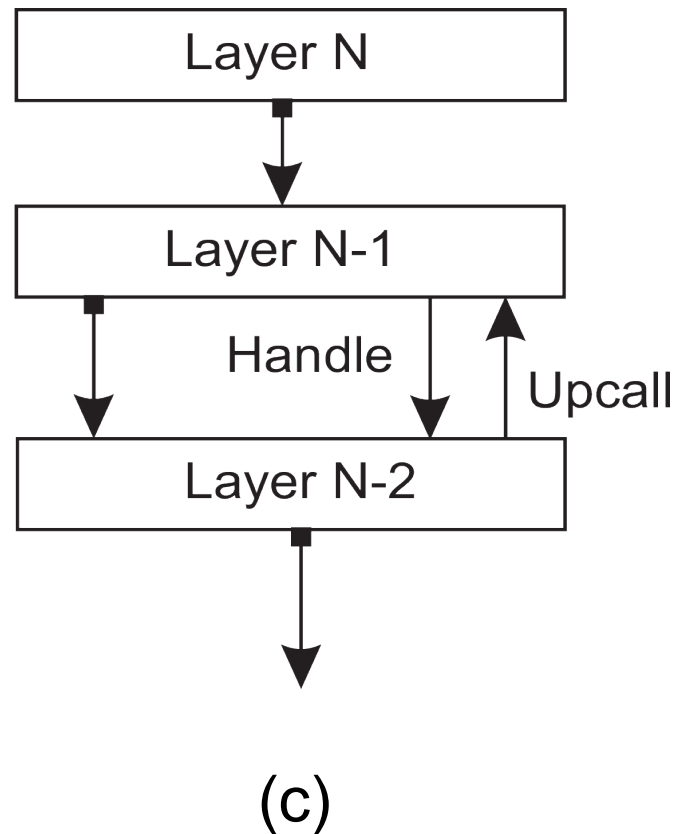


Mixed layered organization

It is organized in a hierarchical manner where each layer provides services to the layer directly above it and acts as a client to the layer directly below. For example, for an application A, At the top layer ($N - 1$), this is the primary application, which directly interacts with the user or other systems. There is a Mathematical Library (L_{math}), Positioned at the middle layer ($N - 2$), this library provides specialized mathematical functionalities that are utilized by Application A. Meanwhile, there is an operating system interface library (L_{os}), At the bottom layer ($N - 3$), this library acts as an interface to the operating system, offering foundational services needed by the mathematical library and potentially other applications. Application A uses L_{math} for specific functions, which in turn relies on L_{os} for more fundamental system-level interactions. L_{os} is common to both Application A and L_{math} , serving as a base layer that abstracts the complexities of the operating system interactions.

Layered Architecture

Different layered organizations



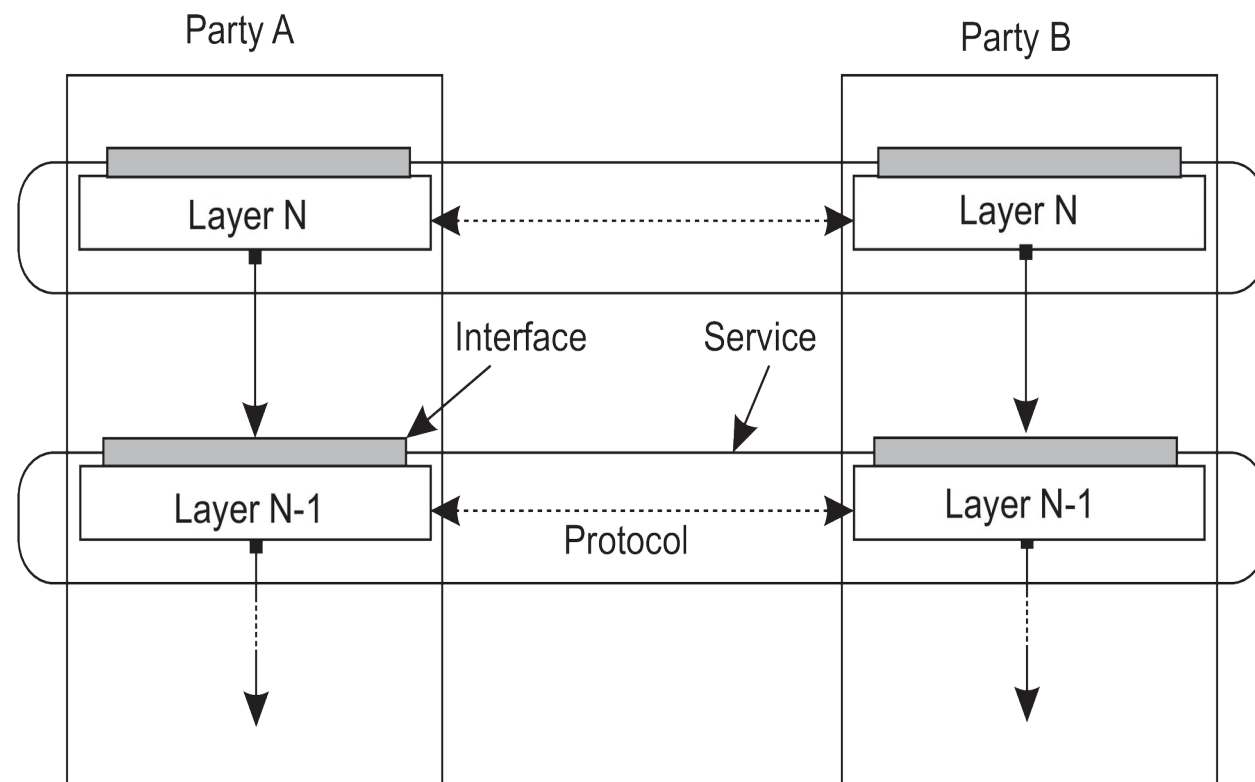
It is convenient to have a lower layer do an upcall to its next higher layer. A typical example is the operating system needs to inform the application about this event so that the application can handle it appropriately (e.g., reading data from a buffer after an I/O completion interrupt).

Layered organization with upcalls

Layered Architecture

- Example: communication protocols

Protocol, service, interface



A well-known and ubiquitously applied layered architecture is that of communication-protocol stacks. In communication-protocol stacks, each layer implements one or several, communication services, allowing data to be sent from a destination to one or several targets. Therefore, each layer offers an interface specifying the functions that can be called.

Another important concept in the case of communication is that of a (communication) protocol, which describes the rules that parties will follow to exchange information. The rules in the Internet is called Transmission Control Protocol, namely TCP. The protocol specifies which messages are to be exchanged for setting up or tearing down a connection, what needs to be done to preserve the ordering of transferred data, and what both parties need to do to detect and correct data that was lost during transmission.

Layered Architecture

- Example: Two-party communication

Server

```
1 from socket import *
2
3 s = socket(AF_INET, SOCK_STREAM)
4 (conn, addr) = s.accept() # returns new socket and addr. client
5 while True:               # forever
6     data = conn.recv(1024) # receive data from client
7     if not data: break     # stop if client stopped
8     msg = data.decode()+"*" # process the incoming data into a response
9     conn.send(msg.encode()) # return the response
10    conn.close()           # close the connection
```

Client

```
1 from socket import *
2
3 s = socket(AF_INET, SOCK_STREAM)
4 s.connect((HOST, PORT)) # connect to server (block until accepted)
5 msg = "Hello World"    # compose a message
6 s.send(msg.encode())    # send the message
7 data = s.recv(1024)    # receive the response
8 print(data.decode())    # print the result
9 s.close()              # close the connection
```

A server is created that makes use of a connection-oriented service as offered by the socket library available in Python. This service allows two communicating parties to reliably send and receive data over a connection.

The combination of constants `AF_INET` and `SOCK_STREAM` is used to specify that the TCP protocol should be used in the communication between the two parties. These two constants can be seen as part of the interface, whereas making use of TCP is part of the offered service.

- `socket()`: to create an object representing the connection
- `accept()`: a blocking call to wait for incoming connection requests; if successful, the call returns a new socket for a separate connection
- `connect()`: to set up a connection to a specified party
- `close()`: to tear down a connection
- `send()`, `recv()`: to send and receive data over a connection, respectively

Layered Architecture

● Application Layering

Traditional three-layered view

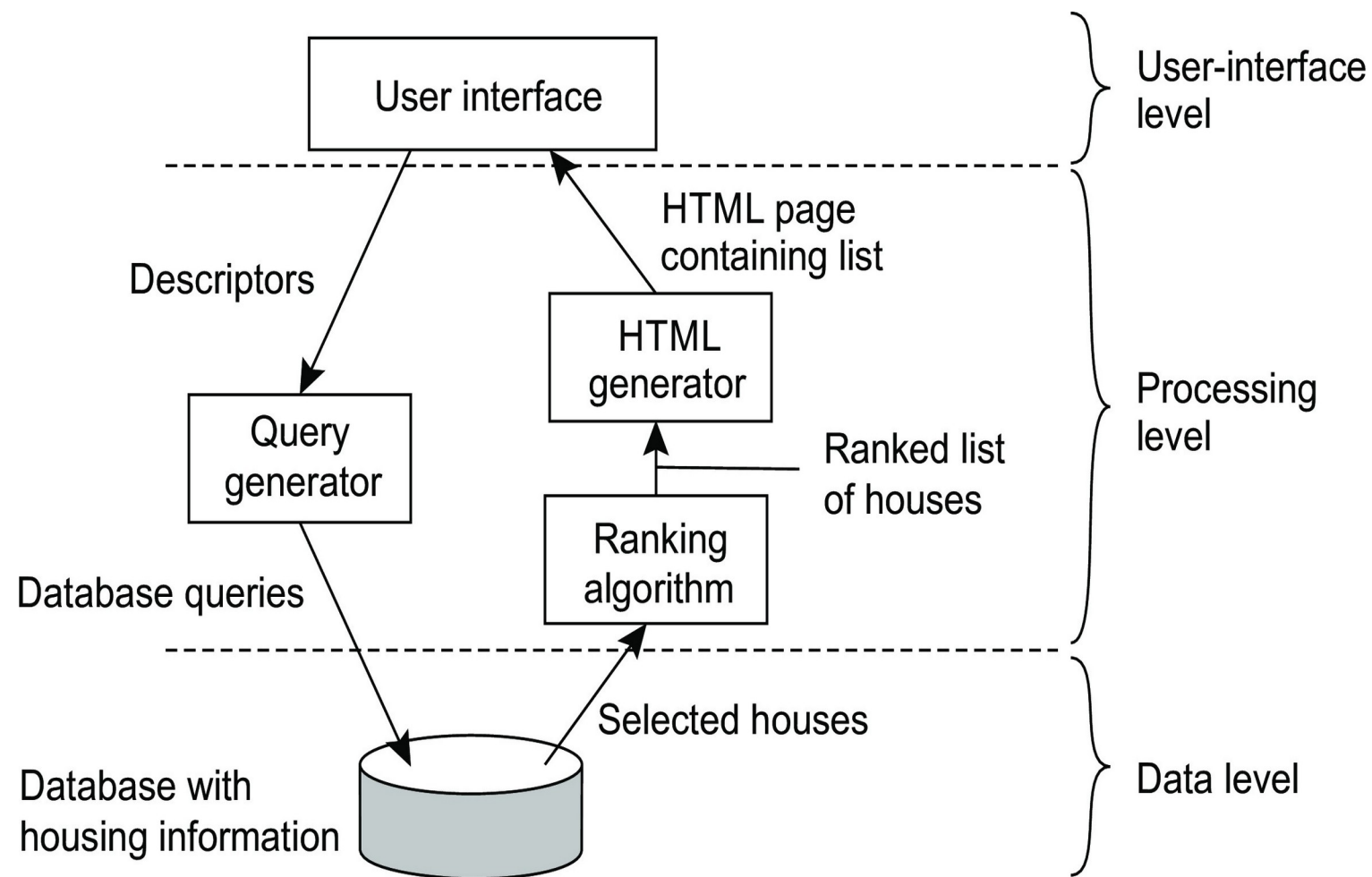
- **Application-interface layer** contains units for interfacing to users or external applications
- **Processing layer** contains the functions of an application, i.e., without specific data
- **Data layer** contains the data that a client wants to manipulate through the application components

Observation

This layering is found in many distributed information systems, using traditional database technology and accompanying applications.

Architectural Styles: Application Layering

Example: a simple search engine



We consider a simple Internet search engine, for example one dedicated to buying houses. Such search engines appear as seemingly simple Websites through which someone can provide descriptors such as a city or region, a price range, the type of house, etc. The back end consists of a huge database of houses currently for sale. The processing layer does nothing else but transform the provided descriptors into a collection of database queries, retrieves the answers and post-processes these answers by, for example, ranking the output by relevance and subsequently generating an HTML page.

Layered Architecture

- Although the layered architectural style is popular, it does have some significant drawbacks, one of the major drawbacks is the strong dependency between different layers.
- A good example of where these dependencies have been carefully managed is in the design of communication protocol stacks. In contrast, a bad example would be applications that have been put together using existing components without much attention to the stability of interfaces or the components themselves, not to mention the overlap in functionality between different components.
- This situation, where there are direct dependencies on specific components, has led to an architectural style that prefers a looser organization into a collection of separate, independent entities. Each entity provides a service. These entities, whether called services, objects, or microservices, all share the common characteristic of being executed as separate processes or threads.
- It's important to note, that running these separate entities doesn't necessarily reduce dependencies compared to the layered architectural style.

Service-Oriented Architecture

- Definition

- A Service-Oriented Architecture or SOA is a design pattern which is designed to build distributed systems that deliver services to other applications through the protocol. It is not limited to any programming language or platform.

- What is Service?

- A service is a well-defined, self-contained function that represents a unit of functionality. A service can exchange information from another service. It is not dependent on the state of another service. It uses a loosely coupled, message-based communication model to communicate with applications and other services.

Service-Oriented Architecture

● Core Principles

- **Loose Coupling:** It is the concept typically employed to deal with the requirements of scalability, flexibility, and fault tolerance. The aim of loose coupling is to minimize dependencies. When there are fewer dependencies, modifications to or faults in one system will have fewer consequences on other systems.
- **Reusability:** Services are designed to be reused in different scenarios, promoting efficiency and consistency across a system.
- **Autonomy:** Services are independent and control their own functionality.

Service-Oriented Architecture

- Service Connections

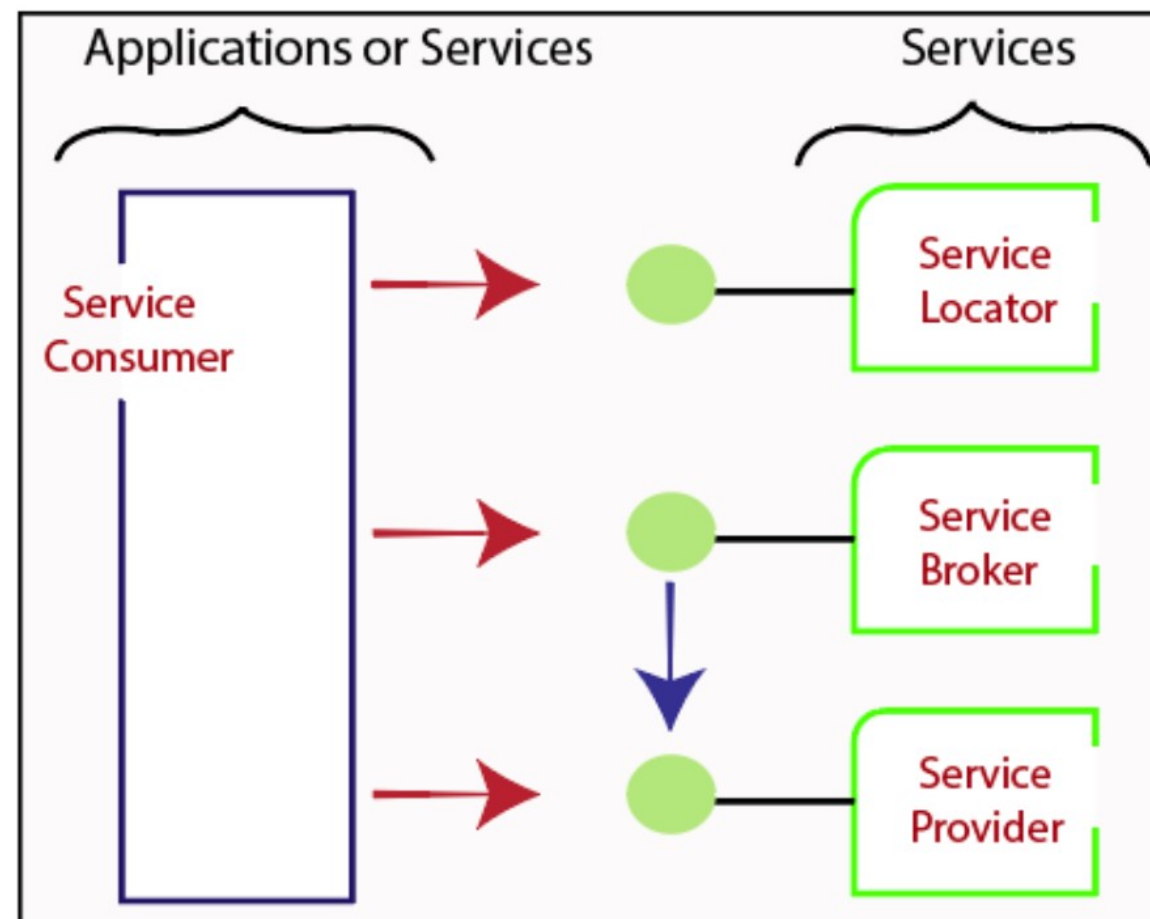
- The figure given below illustrates the service-oriented architecture. Service consumer sends a service request to the service provider, and the service provider sends the service response to the service consumer. The service connection is understandable to both the service consumer and service provider.



Service-Oriented Architecture

• Terminologies

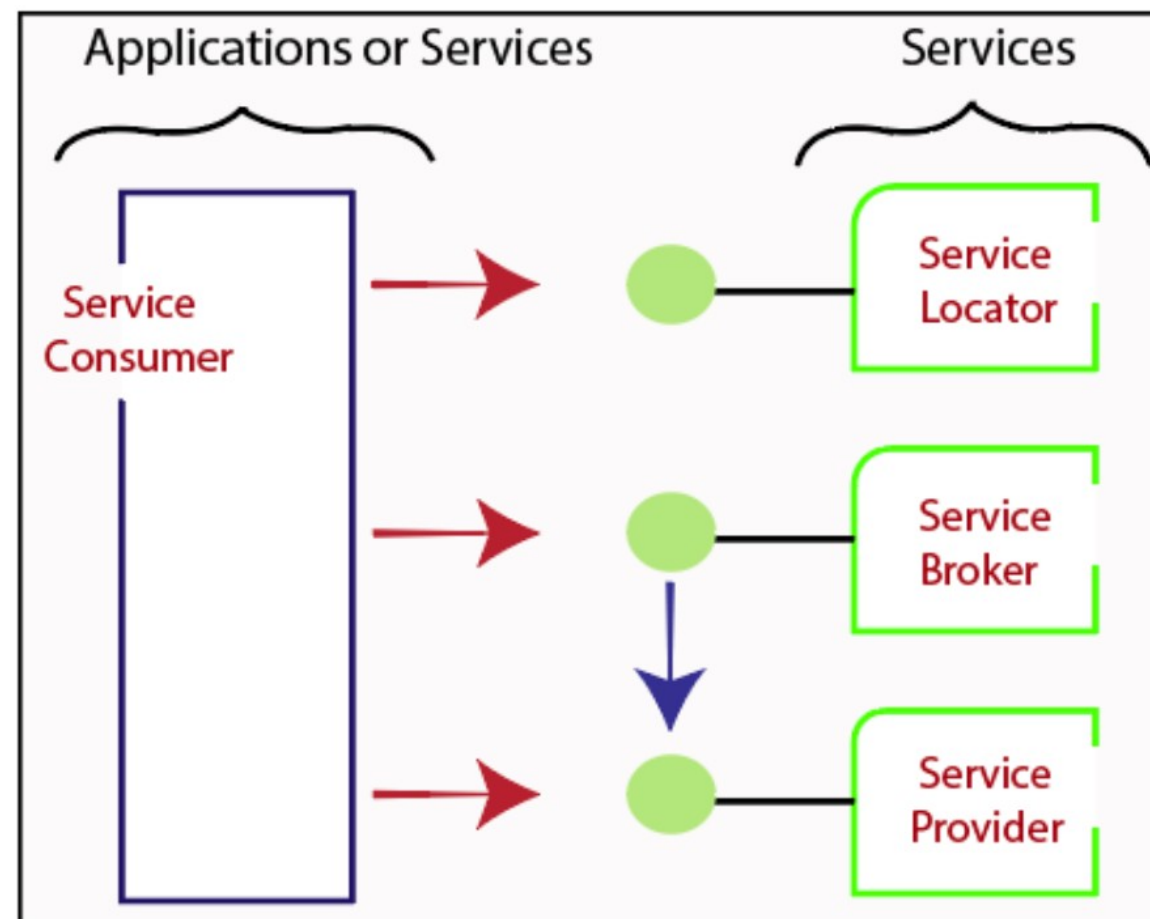
- **Service provider:** It is a software entity that implements a service specification.
- **Service consumer:** It can be called as a requestor or client that calls a service provider. A service consumer can be another service or an end-user application.



Service-Oriented Architecture

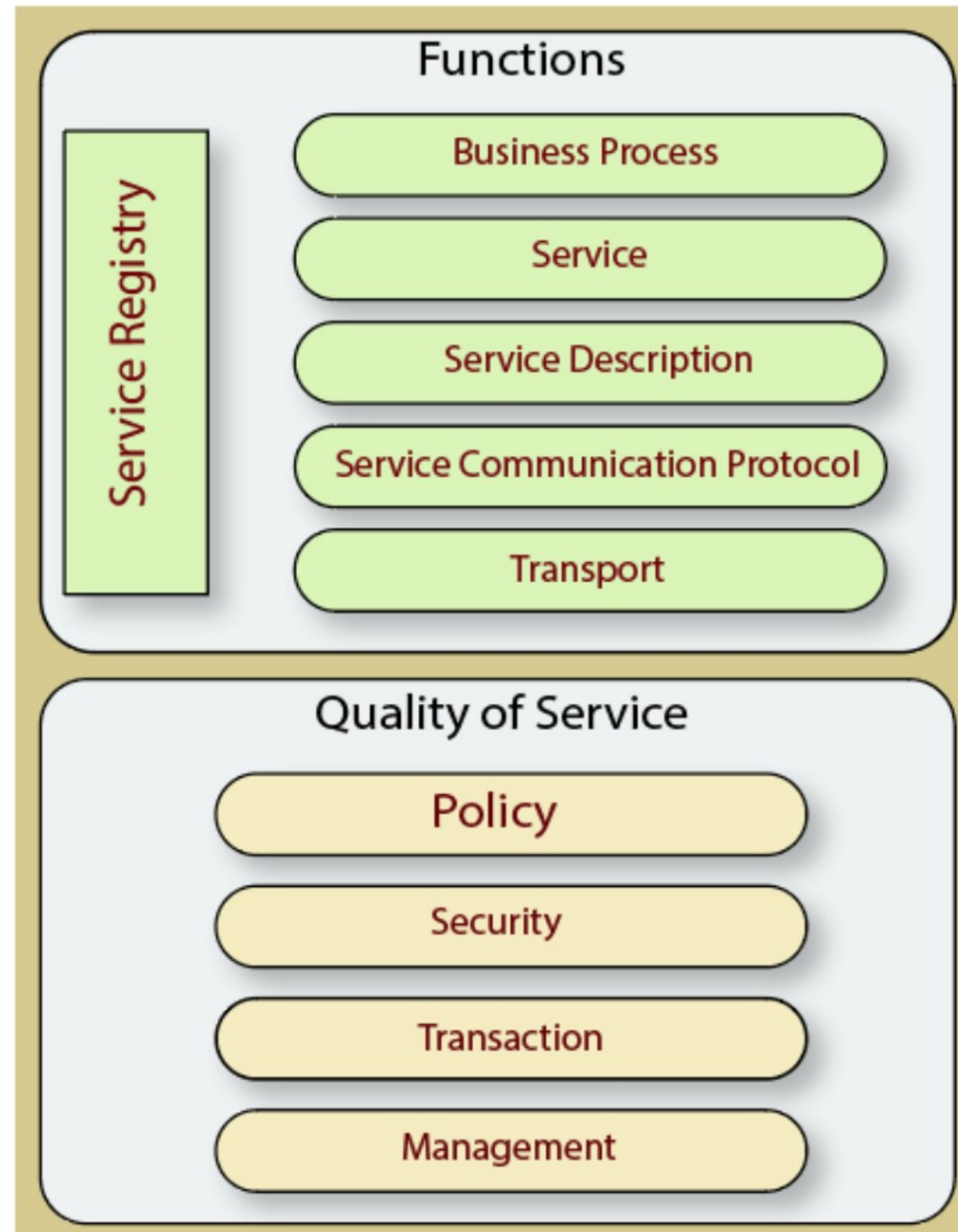
• Terminologies

- **Service locator:** It is a service provider that acts as a registry. It is responsible for examining service provider interfaces and service locations.
- **Service broker:** It is a service provider that pass service requests to one or more additional service providers.



Service-Oriented Architecture

- Components: Functional Aspects and Quality of Service Aspects



Service-Oriented Architecture

● Functional Aspects

- **Transport:** It transports the service requests from the service consumer to the service provider and service responses from the service provider to the service consumer.
- **Service Communication Protocol:** It allows the service provider and the service consumer to communicate with each other.
- **Service Description:** It describes the service and data required to invoke it.
- **Service:** It is an actual service.
- **Business Process:** It represents the group of services called in a particular sequence associated with the particular rules to meet the business requirements.
- **Service Registry:** It contains the description of data which is used by service providers to publish their services.

Service-Oriented Architecture

● Quality of Service Aspects

- **Policy:** It represents the set of protocols according to which a service provider make and provide the services to consumers.
- **Security:** It represents the set of protocols required for identification and authorization.
- **Transaction:** It provides the surety of consistent result. This means, if we use the group of services to complete a business function, either all must complete or none of the complete.
- **Management:** It defines the set of attributes used to manage the services.

Service-Oriented Architecture

● Communication

- Services communicate using standard protocols such as HTTP, REST, or SOAP.
- Messages are often formatted in XML or JSON.
- Communication can be synchronous (request/response model) or asynchronous (message queues).

● Advantages

- **Scalability:** Services can be scaled independently, allowing for more efficient resource use.
- **Flexibility:** Easier to update and modify parts of the system without affecting others.
- **Interoperability:** Because services use standard protocols, they can operate across different technologies and platforms.

Service-Oriented Architecture

● Challenges

- Complexity: Managing interactions and integrations between services can be complex.
- Performance: Network latency and message processing can affect performance.
- Security: Exposing services over a network increases vulnerability to security threats.

● Use Case: Integrating Banking Systems

- Banks typically operate a variety of applications and systems for handling different financial products and services such as loans, checking and savings accounts, and investments

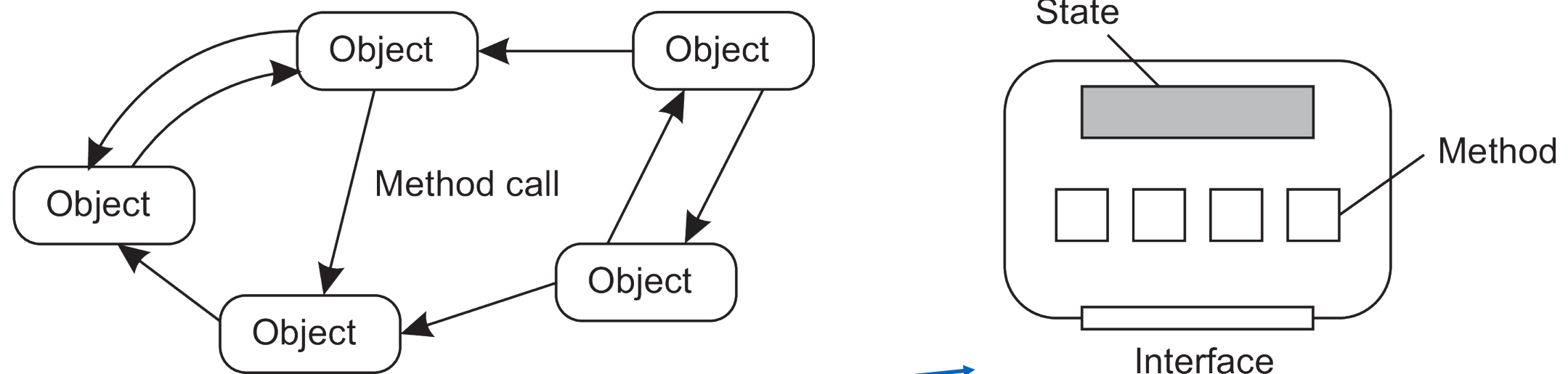


Service-Oriented Architecture

- Object-based architectural style

Essence

Components are objects, connected to each other through procedure calls. Objects may be placed on different machines; calls can thus execute across a network.



Encapsulation

Objects are said to **encapsulate data** and offer **methods on that data** without revealing the internal implementation.

The object-based approach in distributed systems allows for a flexible and encapsulated way to manage components, making it easier to handle data and operations across different machines seamlessly.

Service-Oriented Architecture

- Resource-based architectural style (**Re**presentational **S**tate **T**ransfer, REST)
Essence

View a distributed system as a collection of resources, individually managed by components. Resources may be added, removed, retrieved, and modified by (remote) applications.

1. Resources are identified through a single naming scheme
2. All services offer the same interface
3. Messages sent to or from a service are fully self-described
4. After executing an operation at a service, that component forgets everything about the caller

Basic operations

Operation	Description
PUT	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource
POST	Modify a resource by transferring a new state

Service-Oriented Architecture

- Resource-based architectural style (**R**epresentational **S**tate **T**ransfer, REST)

Example: Amazon's Simple Storage Service (Amazon S3)

Essence

Objects (i.e., files) are placed into **buckets** (i.e., directories). Buckets cannot be placed into buckets. Operations on `ObjectName` in bucket `BucketName` require the following Uniform Resource Identifier (URI):

<http://BucketName.s3.amazonaws.com/ObjectName>

Typical operations

All operations are carried out by sending HTTP requests:

- Create a bucket/object: `PUT`, along with the URI, an application sends a `PUT` request with the URI of the bucket or object, If the bucket or object already exists, an HTTP error message is returned.
- Listing objects: `GET` on a bucket name, to find out which objects are in a bucket, an application sends a `GET` request with the URI of that bucket
- Reading an object: `GET` on a full URI

Service-Oriented Architecture

- Resource-based architectural style (**R**epresentational **S**tate **T**ransfer, REST)
 - The REST architecture has gained popularity because of its simplicity. It uses standard HTTP request (GET, PUT, POST, DELETE), making it straightforward and easy to understand. However, this simplicity can sometimes be a limitation. For example, in scenarios requiring complex communication schemes, such as distributed transactions, REST architecture may fall short because it generally requires services to maintain the state of execution [but REST forgets everything after executing an operation at a service].
 - Nevertheless, there are many instances where REST architectures match well, offering a simple integration scheme for services without coming with numerous service-specific interfaces. This balance of simplicity and functionality is what makes REST architecture both powerful and easy to use in many web-based distributed systems.

Publish-Subscribe Architecture

- Coordination

Our systems keep getting bigger and more dynamic, where processes can join or leave easily, we need an architecture with loose dependencies between processes. A lot of distributed systems have adopted an approach where there's a clear separation between processing and coordination. Think of a system as a bunch of processes that operate independently. In this model, coordination is all about how these processes communicate and work together. It's like the glue that holds everything together.

Two coordination models: Temporal and referential coupling

	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

When processes are both temporally and referentially coupled, they coordinate directly. This means they need to know each other and be active at the same time, such as making a phone call—you need to know the number and the other person needs to pick up.

There's mailbox coordination. Here, processes are temporally decoupled but referentially coupled. They don't need to be active simultaneously. Instead, they drop messages in a mailbox. However, they need to know where the mailbox is.

Publish-Subscribe Architecture

Two coordination models: Temporal and referential coupling

- Coordination

	Temporally coupled	Temporally decoupled
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

Next, we have event-based coordination, which happens when processes are temporally coupled but referentially decoupled. The processes don't know each other directly. They just publish notifications about events, like "Hey, I did something cool!" Other processes can subscribe to these notifications. Ideally, notifications reach only those who subscribed to them, but subscribers need to be active when the notification is sent.

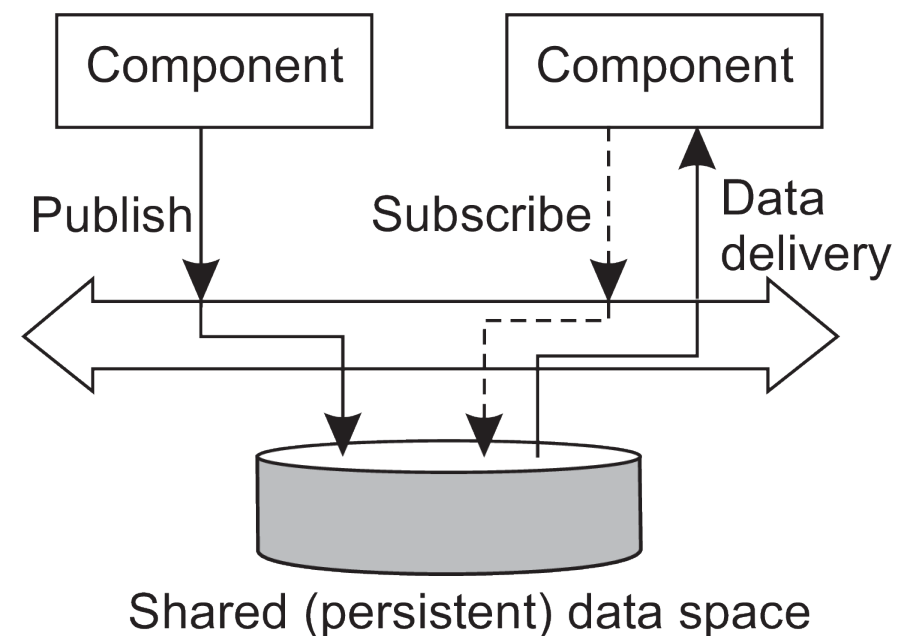
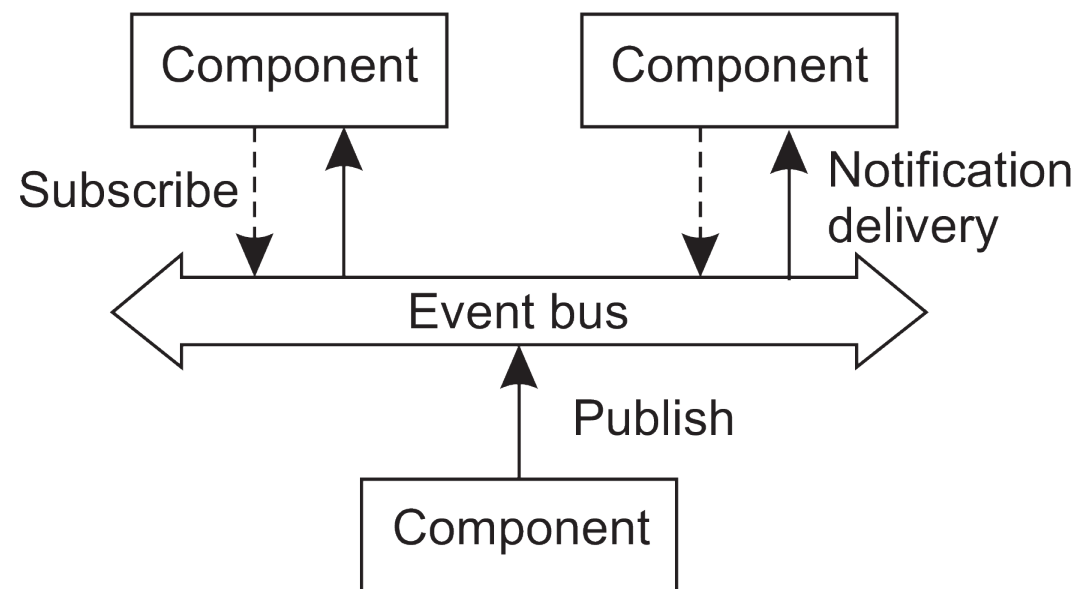
A well-known model is when processes are both temporally and referentially decoupled, known as a shared data space. Here, processes communicate through tuples—structured data records similar to database rows. A process can put tuples into the shared data space and retrieve them using a search pattern.

Shared data spaces also support associative searches. When a process wants to get a tuple, it specifies some values of the fields it's interested in. Any matching tuple is returned.

Publish-Subscribe Architecture

- Coordination

Event-based and Shared data space



- Shared data spaces are combined with event-based coordination. A process subscribes to certain tuples by giving a search pattern, and when a new matching tuple is added, the subscriber is notified. This is a publish-subscribe architecture, where processes don't directly know each other.

Publish-Subscribe Architecture

- Example: Linda tuple space

Three simple operations

- `in(t)`: remove a tuple matching template `t`
- `rd(t)`: obtain copy of a tuple matching template `t`
- `out(t)`: add tuple `t` to the tuple space

More details

- Calling `out(t)` twice in a row, leads to storing **two** copies of tuple `t` $\Rightarrow$ a tuple space is modeled as a **multiset**.
- Both `in` and `rd` are **blocking** operations: the caller will be blocked until a matching tuple is found, or has become available.

Publish-Subscribe Architecture

● Example: Linda tuple space

<string, string, string>

<poster's name, topic, message>

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("bob","distsys","I am studying chap 2"))
4 blog._out(("bob","distsys","The linda example's pretty simple"))
5 blog._out(("bob","gtcn","Cool book!"))
```

(a) Bob's code for creating a microblog and posting three messages.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("alice","gtcn","This graph theory stuff is not easy"))
4 blog._out(("alice","distsys","I like systems more than graphs"))
```

(b) Alice's code for creating a microblog and posting two messages.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 t1 = blog._rd(("bob","distsys",str))
4 t2 = blog._rd(("alice","gtcn",str))
5 t3 = blog._rd(("bob","gtcn",str))
```

(c) Chuck reading a message from Bob's and Alice's microblog.

- Messages are tagged with the poster's name, a topic, and the message content. Each message is represented as a tuple <string, string, string>, where the first string is the poster's name, the second is the topic, and the third is the actual message. Imagine we have a shared data space called MicroBlog. This shows how Alice and Bob can post messages to this space, and how Chuck can pick a randomly selected message. Notice that neither Alice nor Bob knows who will read their posts.

Publish-Subscribe Architecture

● Example: Linda tuple space

<string, string, string>

<poster's name, topic, message>

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("bob","distsys","I am studying chap 2"))
4 blog._out(("bob","distsys","The linda example's pretty simple"))
5 blog._out(("bob","gtcn","Cool book!"))
```

(a) Bob's code for creating a microblog and posting three messages.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("alice","gtcn","This graph theory stuff is not easy"))
4 blog._out(("alice","distsys","I like systems more than graphs"))
```

(b) Alice's code for creating a microblog and posting two messages.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 t1 = blog._rd(("bob","distsys",str))
4 t2 = blog._rd(("alice","gtcn",str))
5 t3 = blog._rd(("bob","gtcn",str))
```

(c) Chuck reading a message from Bob's and Alice's microblog.

- In the first line of each code fragment, a process looks up the tuple space named "MicroBlog." Bob posts three messages: two on the topic "distsys" and one on "gtcn." Alice posts two messages, one on each topic. Finally, Chuck reads three messages: one from Bob on "distsys," one from Bob on "gtcn," and one from Alice on "gtcn."

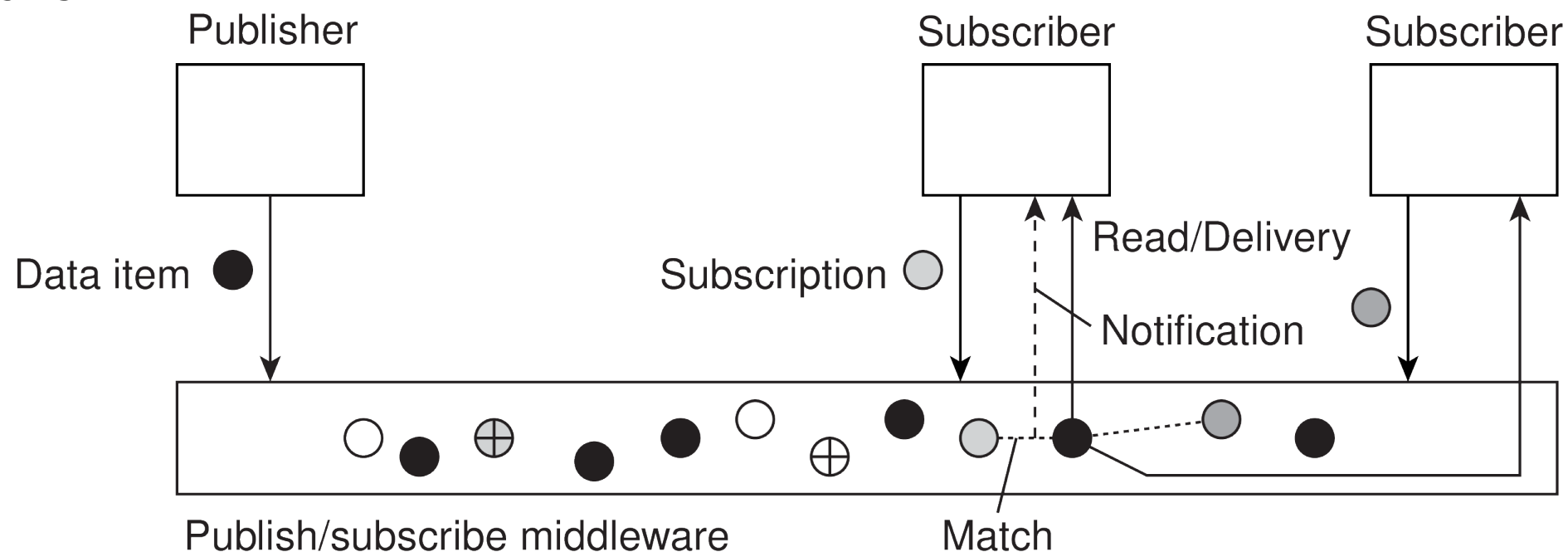
Publish-Subscribe Architecture

Issue: how to match events?

- One key aspect of publish-subscribe systems is that communication happens based on the events that a subscriber is interested in.
- Assume events are described by (attribute, value) pairs
- When a notification about an event is published, it becomes available for other processes to read. To make this happen, a subscription needs to be sent to the middleware, which includes a description of the event that the subscriber is interested in.
- **topic-based subscription**: specify a “attribute = value” series
- **content-based subscription**: the attribute is expected to have values within a specific range, specify a “attribute $\in$ range” series. Obviously, the more detailed the description, the harder it is to match an event to it.

Publish-Subscribe Architecture

- We need to match subscriptions against notifications.
- An event corresponds to data becoming available. When a match is found, there are two possible scenarios. In the first case, the middleware might forward the notification and the associated data to all subscribers with matching subscriptions. Alternatively, the middleware might only send the notification, and subscribers then execute a read operation to get the data.
- If the data is immediately forwarded with the notification, the middleware usually doesn't store the data.
- If subscribers need to explicitly read the data after receiving a notification, the middleware has to store the data. This setup involves additional data management operations.



Publish-Subscribe Architecture

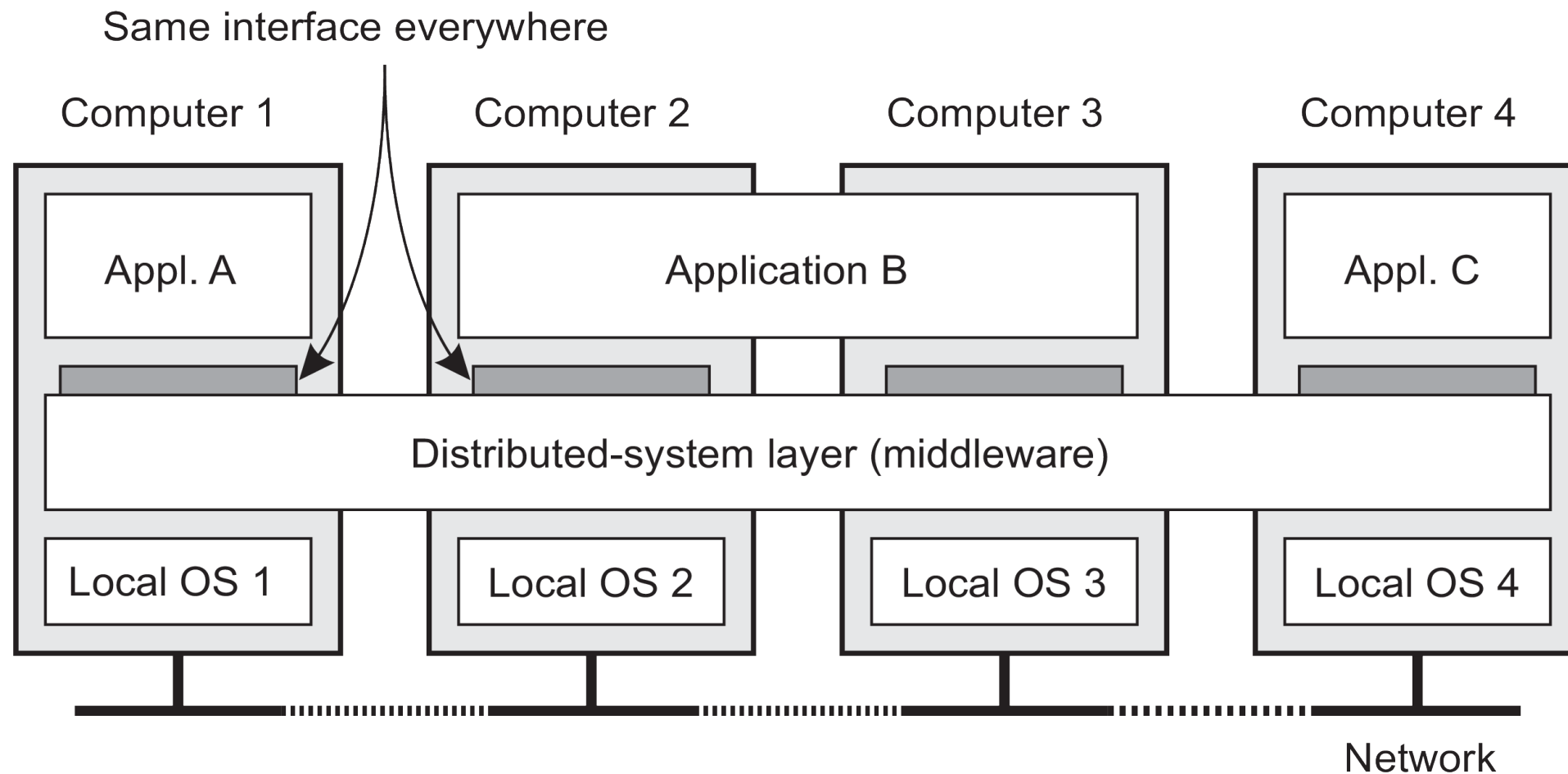
- Events can complicate subscription processing. For example, consider a subscription like "notify when room 1 is unoccupied and the door is unlocked." This requires independent sensors for room occupancy (like motion sensors) and door lock status. These primitive events need to be composed into a publishable data item that processes can subscribe to. Event composition is tricky, especially when primitive events come from different parts of the distributed system.

Observation

Content-based subscriptions may easily have serious scalability problems (why?)

In content-based publish-subscribe systems, the complexity of matching detailed and varied subscriptions to incoming notifications can lead to significant performance bottlenecks. Additionally, as the system scales, the computational load and resource demands for matching these subscriptions grow exponentially. This makes it harder to ensure efficient performance, especially when also considering the security and privacy. Therefore, the practical implementation of content-based systems often encounters serious scalability issues.

Middleware: the OS of distributed systems



- Build distributed applications, organize distributed systems with a separate software layer on top of the operating systems of the computers in the network, which is known as middleware.
- Four networked computers and three applications. Application B, for example, runs across computers 2 and 3. Each application gets the same interface. The distributed system allows components of a single distributed application to communicate with each other and lets different applications communicate too. It also hides the differences in hardware and operating systems from each application as much as possible.

Middleware: the OS of distributed systems

- Middleware manages resources and allows applications to share and use those resources efficiently across a network.

What services it offer?

- Facilities for interapplication communication.
- Security services.
- Accounting services.
- Masking of and recovery from failures.

What does it contain?

The key difference is that middleware services operate in a networked environment. These services are useful to many applications. Commonly used components and functions that need not be implemented by applications separately.

Middleware

Using legacy to build middleware

Middleware is that software layer that helps different applications communicate and work together.

Problem

The interfaces offered by a legacy component are most likely not suitable for all applications.

Solution

A **wrapper** or **adapter** offers an interface acceptable to a client application. Its functions are transformed into those available at the component.

Middleware organization

- Wrappers

A wrapper or adapter is a special piece of software that offers an interface suitable for the client application. It transforms the functions of the old component into something the modern application can use. Essentially, it fixes the problem of incompatible interfaces.

Originally, this concept was mostly used in object-oriented programming, but in distributed systems, wrappers are more than just simple interface converters. For example, an object adapter lets applications call remote objects, even if those objects are just a bunch of library functions working on database tables.

Take Amazon's S3 storage service, for instance. It offers two types of interfaces: one that follows a RESTful architecture and another more traditional one. For the RESTful interface, clients use the HTTP protocol to communicate with a traditional web server, which acts as an adapter to the actual storage service. The server processes the requests and hands them off to specialized servers within S3.

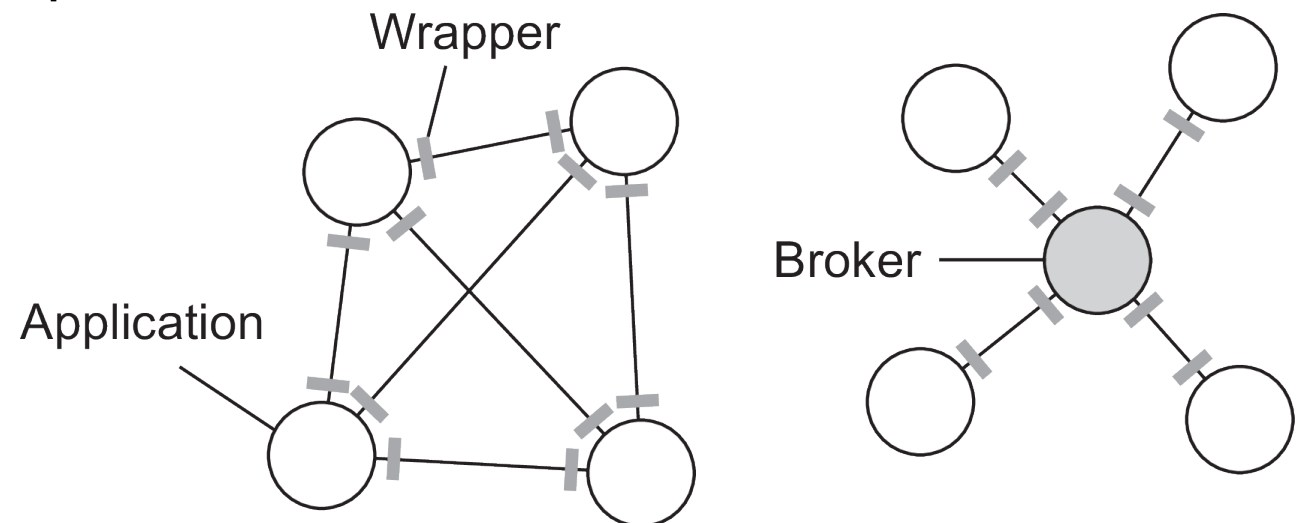
Middleware organization

● Wrappers

Wrappers have always been crucial for integrating existing components into new systems. They are essential for making systems extensible, which is important for achieving openness. In the past, if Application A needed data from Application B, you'd create a specific wrapper for B to access A's data. However, this approach doesn't scale well.

To reduce the number of wrappers needed, we use middleware. One common solution is a broker, a centralized component that manages all interactions between different applications. A common type of broker is a message broker. With a message broker, applications send requests to the broker with information on what they need. The broker, knowing all relevant applications, contacts the appropriate ones, combines and transforms the responses, and sends the result back to the requesting application.

Two solutions: 1-on-1 or through a broker



Complexity with N applications

- 1-on-1: requires $N \times (N - 1) = O(N^2)$ wrappers
- broker: requires $2N = O(N)$ wrappers

Middleware organization

- Interceptors: control flow

In concept, an interceptor is just a piece of software that interrupts the usual flow of control to let some other code run. Interceptors are key for tailoring middleware to fit the specific needs of an application, making the middleware more adaptable. They play a big role in keeping middleware open and flexible.

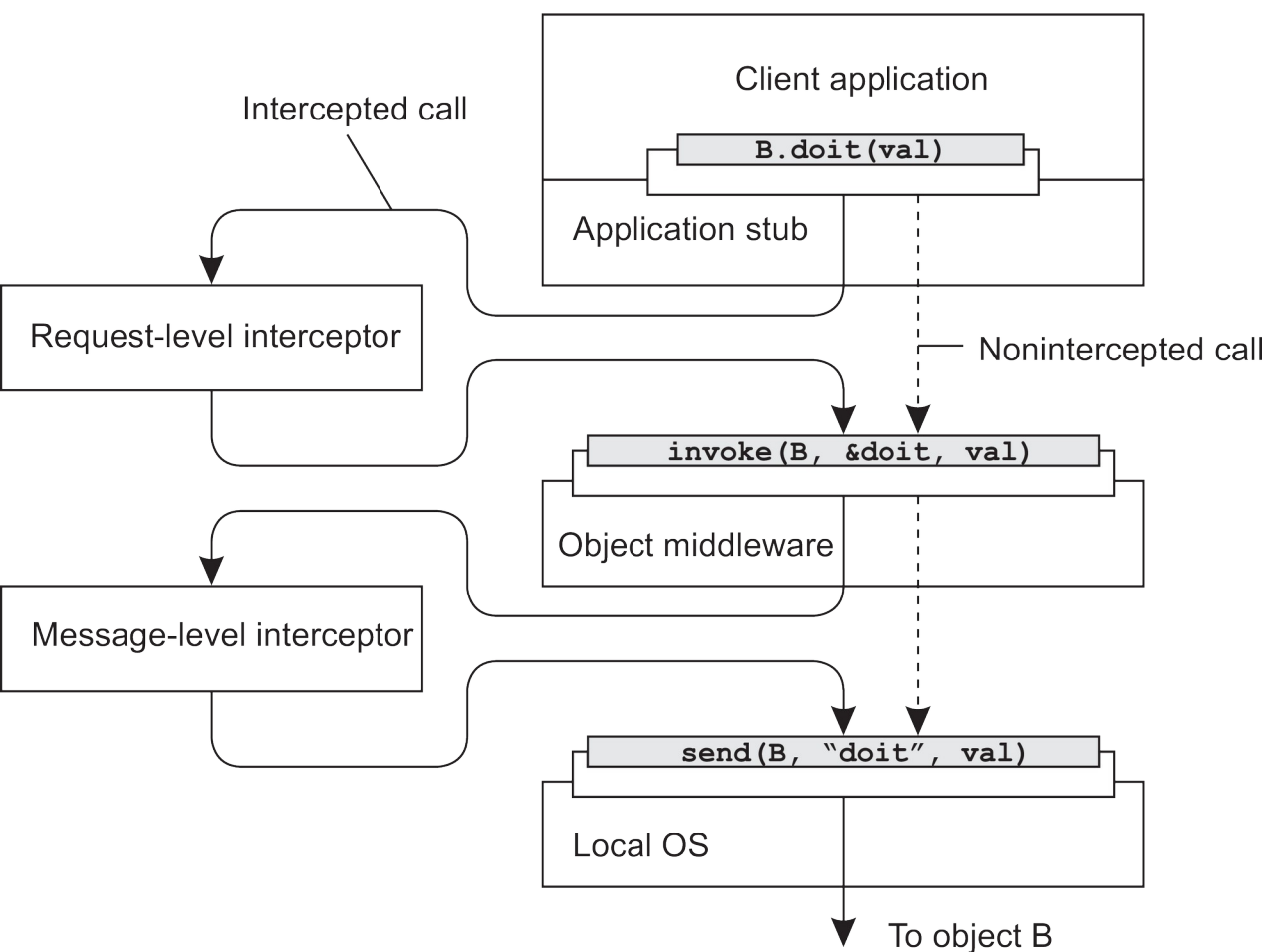
However, making interceptors very generic can require a lot of effort. It's debatable whether it's worth making them super flexible or if it's better to keep them simple and specific. Also, having only limited interception capabilities can make the software and the distributed system easier to manage.

Example: Object A can call a method on Object B, even if B is on a different machine. This remote-object call happens in three steps:

1. Object A has a local interface that looks the same as Object B's interface. A calls a method using this interface.
2. This call is transformed into a generic object invocation, thanks to the general object-invocation interface provided by the middleware on A's machine.
3. Finally, this generic invocation is turned into a message and sent over the network using A's local operating system's transport-level interface.

Middleware organization

- Interceptors: control flow



After the first step, the call `B.doit(val)` becomes a generic call like `invoke(B, &doit, val)` with a reference to B's method and the call parameters. Now, imagine B is replicated. Each replica needs to be invoked, and this is where interception is useful. The request-level interceptor will call `invoke(B, &doit, val)` for each replica. The interesting point here is that Object A doesn't need to know about B's replication, nor does the middleware need special components to handle the replicated call. Only the request-level interceptor needs to know about B's replication.

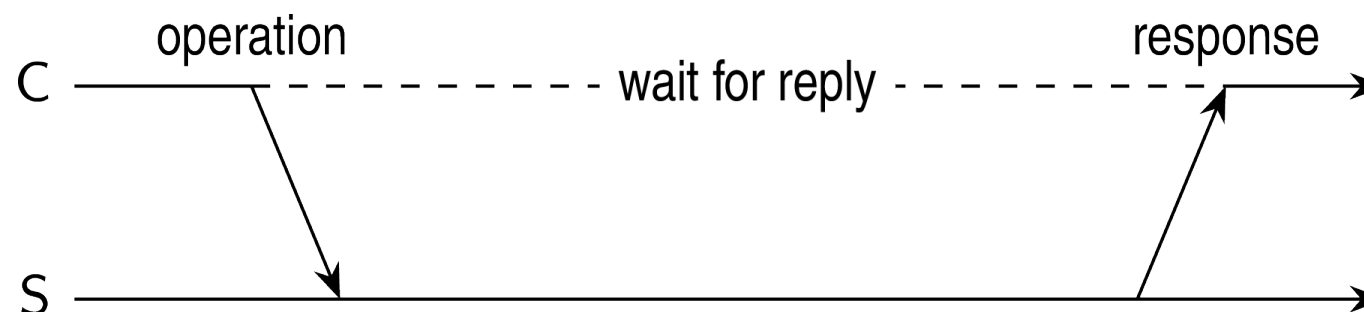
Finally, a call to a remote object has to be sent over the network. This means using the messaging interface provided by the local operating system. At this level, a message-level interceptor can help transfer the invocation to the target object. For example, if the parameter `val` is a huge data array, it might be wise to break it into smaller parts and reassemble it at the destination. This fragmentation can improve performance or reliability. The middleware doesn't need to know about this; the lower-level interceptor takes care of it, handling the communication with the local operating system.

Layered-System

- Client-server architecture

Characteristics:

- In the basic client-server model, processes in a distributed system are split into two (sometimes overlapping) groups. A server is a process that provides a specific service, like a file system service or a database service. A client is a process that requests a service from a server by sending a request and waiting for a reply. This client-server interaction, also known as request-reply behavior.
- Communication between a client and a server can use a simple connectionless protocol when the network is pretty reliable, like in many local-area networks. When a client needs a service, it sends a message to the server with the service it wants and any necessary input data. The server waits for incoming requests, processes them, and sends back the results in a reply message.



Layered-System

- Client-server architecture

Characteristics:

- Using a connectionless protocol is efficient as long as messages don't get lost or corrupted. But making the protocol handle occasional transmission failures is tricky. The client might resend the request if no reply comes back, but it can't tell if the original request was lost or if the reply was lost. If the reply was lost, resending the request might cause the operation to happen twice. For example, if the operation is "transfer \$10,000 from my bank account," repeating it would be a big problem. But if the operation is "tell me how much money I have left," repeating it is fine. There's no one-size-fits-all solution for lost messages.
- Another option is to use a reliable connection-oriented protocol. This isn't great for local-area networks because it can be slow, but it works well in wide-area systems where communication is less reliable. Most Internet application protocols use reliable TCP/IP connections. In this case, the client sets up a connection to the server before sending a request. The server uses the same connection to send the reply, and then the connection is closed. However, setting up and tearing down a connection can be costly, especially for small requests and replies.

Layered-System: Centralized System Architectures

● Basic Client – Server Model

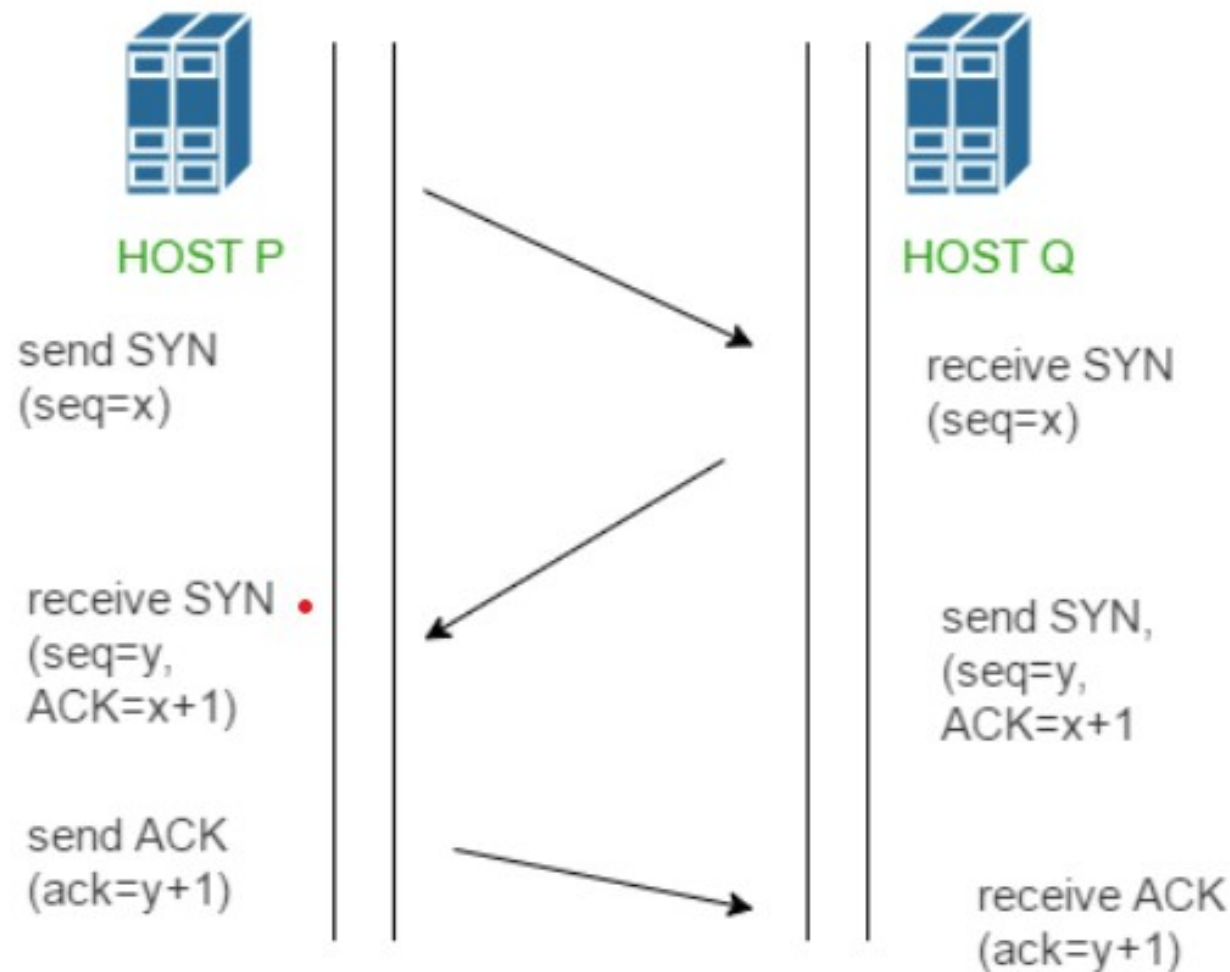
- **Advantages:** It has high efficiency, as long as messages do not get lost or corrupted, the request/reply works fine.
- **Disadvantages:** Making the protocol resistant to occasional transmission failures is not trivial. The only thing we can do is possibly let the client resend the request when no reply message comes in. The client cannot detect whether the original request message was lost, or that transmission of the reply failed.
- **Solution:** TCP 3-Way Handshake Process

TCP provides reliable communication with something called Positive Acknowledgement with Re-transmission(PAR). The Protocol Data Unit(PDU) of the transport layer is called a segment. Now a device using PAR resend the data unit until it receives an acknowledgement. If the data unit received at the receiver's end is damaged, the receiver discards the segment. So the sender has to resend the data unit for which positive acknowledgement is not received.

Layered-System: Centralized System Architectures

● 3 – Way Handshake Steps:

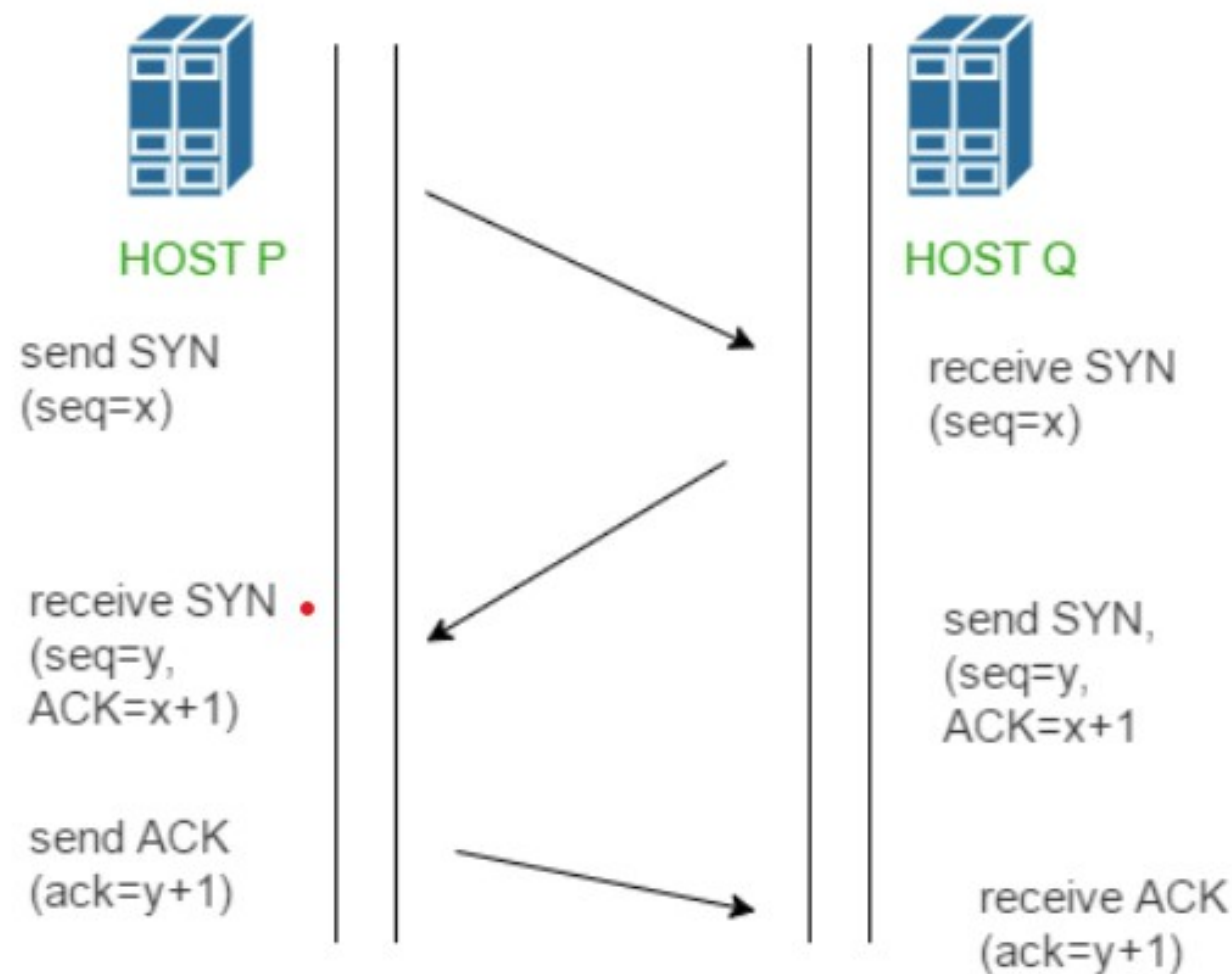
- **Step 1 (SYN):** In the first step, the client wants to establish a connection with a server, so it sends a segment with SYN (Synchronize Sequence Number) which informs the server that the client is likely to start communication and with what sequence number it starts segments with.



Layered-System: Centralized System Architectures

- 3 – Way Handshake Steps:

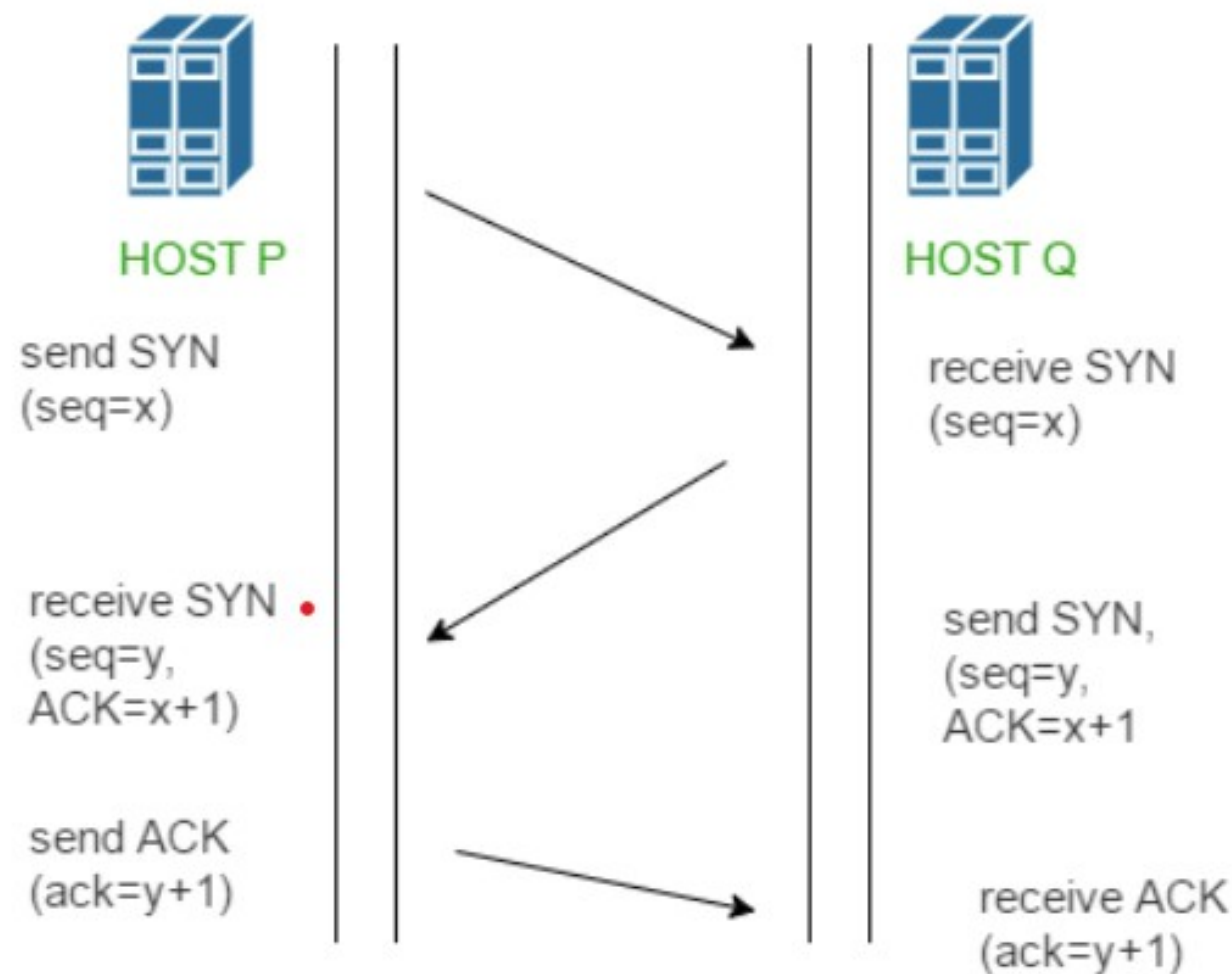
- **Step 2 (SYN + ACK):** Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of the segment it received and SYN signifies with what sequence number it is likely to start the segments with.



Layered-System: Centralized System Architectures

- 3 – Way Handshake Steps:

- **Step 3 (ACK):** In the final part client acknowledges the response of the server and they both establish a reliable connection with which they will start the actual data transfer.



Layered-System

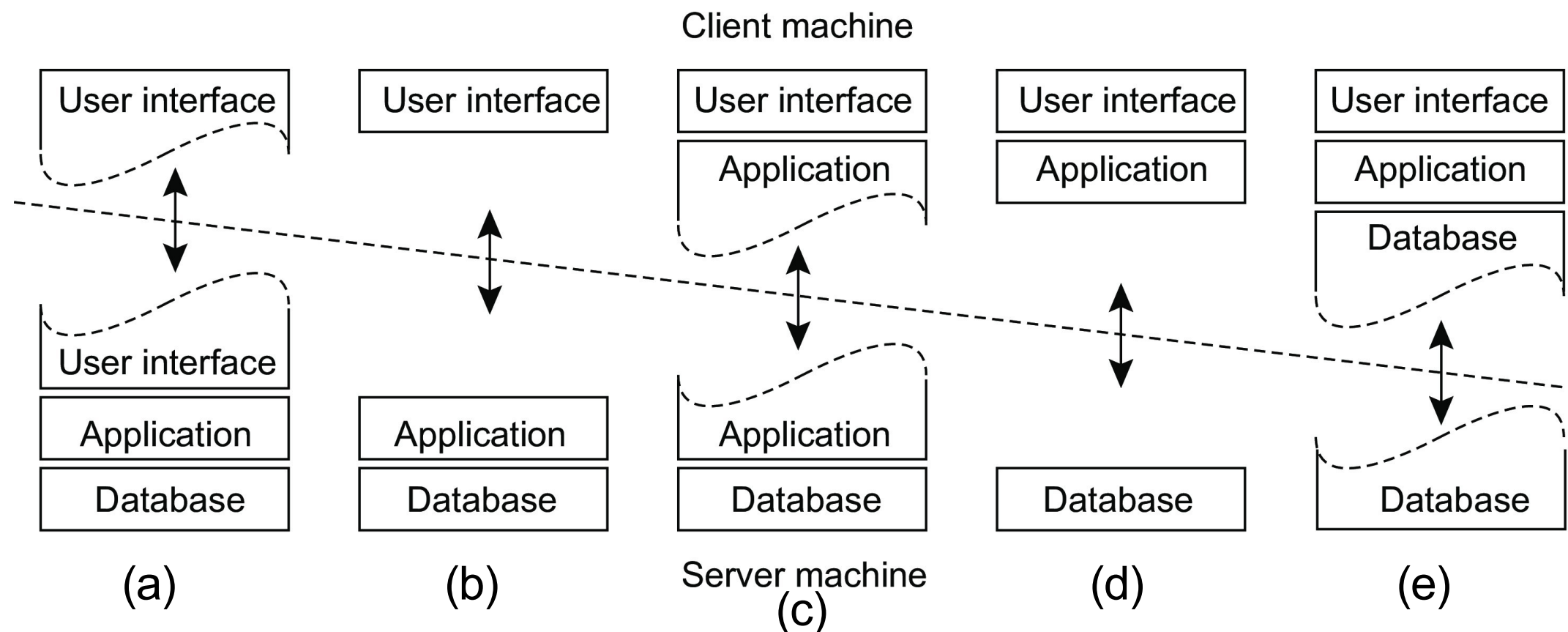
- Multitiered system architectures

Some traditional organizations

- **Single-tiered:** dumb terminal/mainframe configuration, the server does most of the work
- **Two-tiered:** client/single server configuration
- **Three-tiered:** each layer on separate machine

Traditional two-tiered configurations

Many distributed applications are divided into three layers: the user-interface layer, the processing layer, and the data layer. One way to set up clients and servers is to spread these layers across different machines

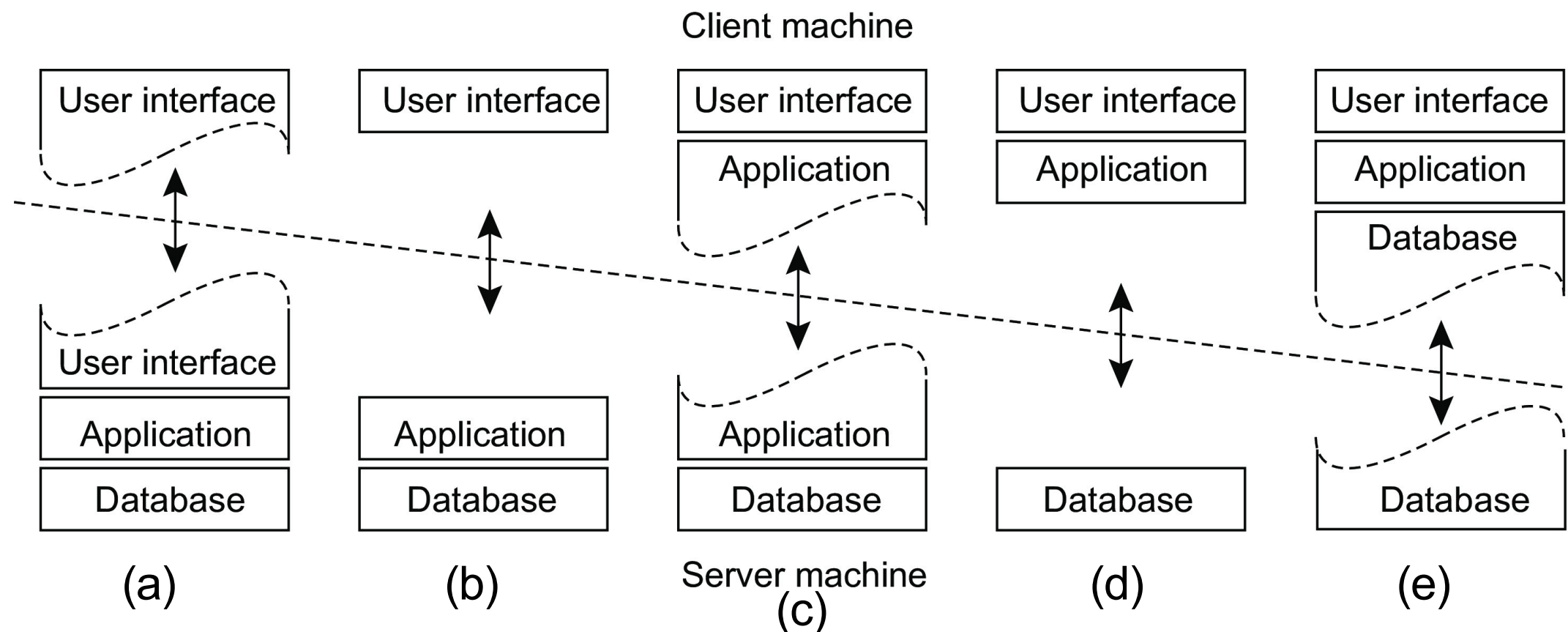


Layered-System

- Multitiered system architectures

Traditional two-tiered configurations

- Only have terminal-dependent part of the user interface on the client machine. The server handles the rest.
- Put the entire user-interface software on the client side. The client has a graphical front end that talks to the rest of the application on the server through a specific protocol. The client only handles the interface, no processing.

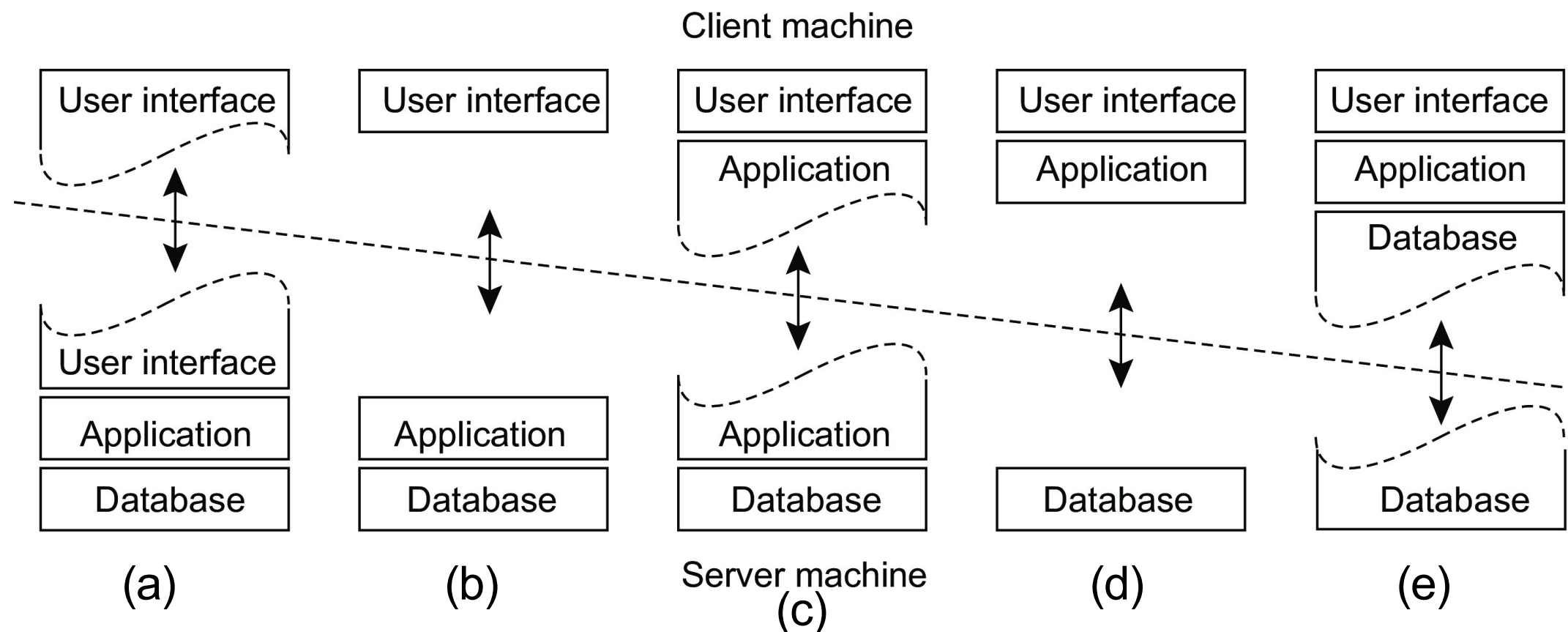


Layered-System

- Multitiered system architectures

Traditional two-tiered configurations

(c) Move part of the application to the front end. This makes sense for applications that use forms that need to be filled out before processing. The front end can check the form's correctness and consistency and interact with the user if needed. Another example is a word processor where basic editing happens on the client, but advanced features like spell check run on the server.



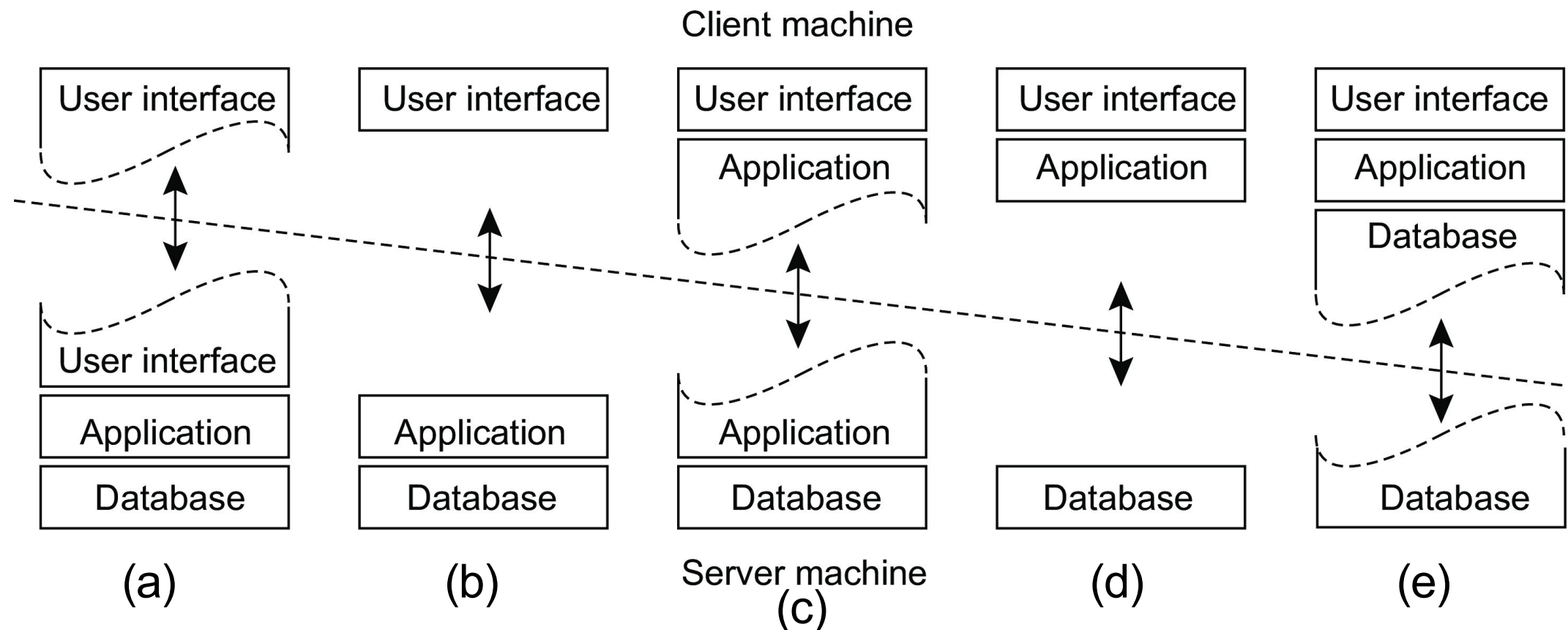
Layered-System

- Multitiered system architectures

Traditional two-tiered configurations

(d) The client is a PC or workstation connected to a distributed file system or database. Most of the application runs on the client, but file and database operations go to the server. For example, many banking apps run on the user's machine where transactions are prepared. Once ready, the app contacts the bank's database server to upload the transactions for processing.

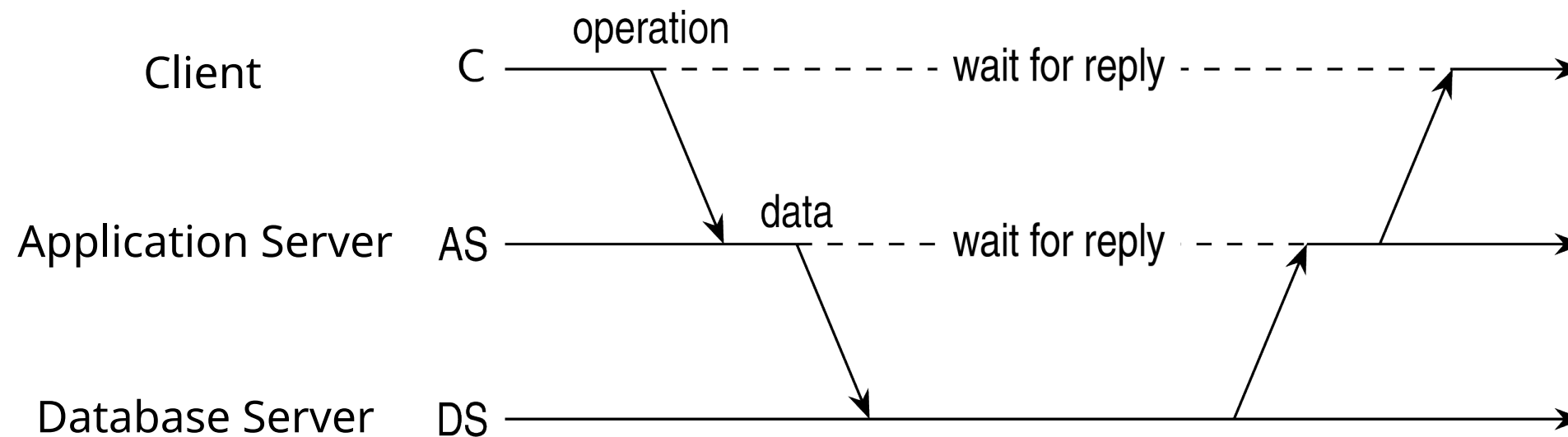
(e) The client's local disk stores part of the data. For instance, when browsing the web, the client builds a cache on the local disk of recently viewed web pages.



Layered-System

Being client and server at the same time

Three-tiered architecture



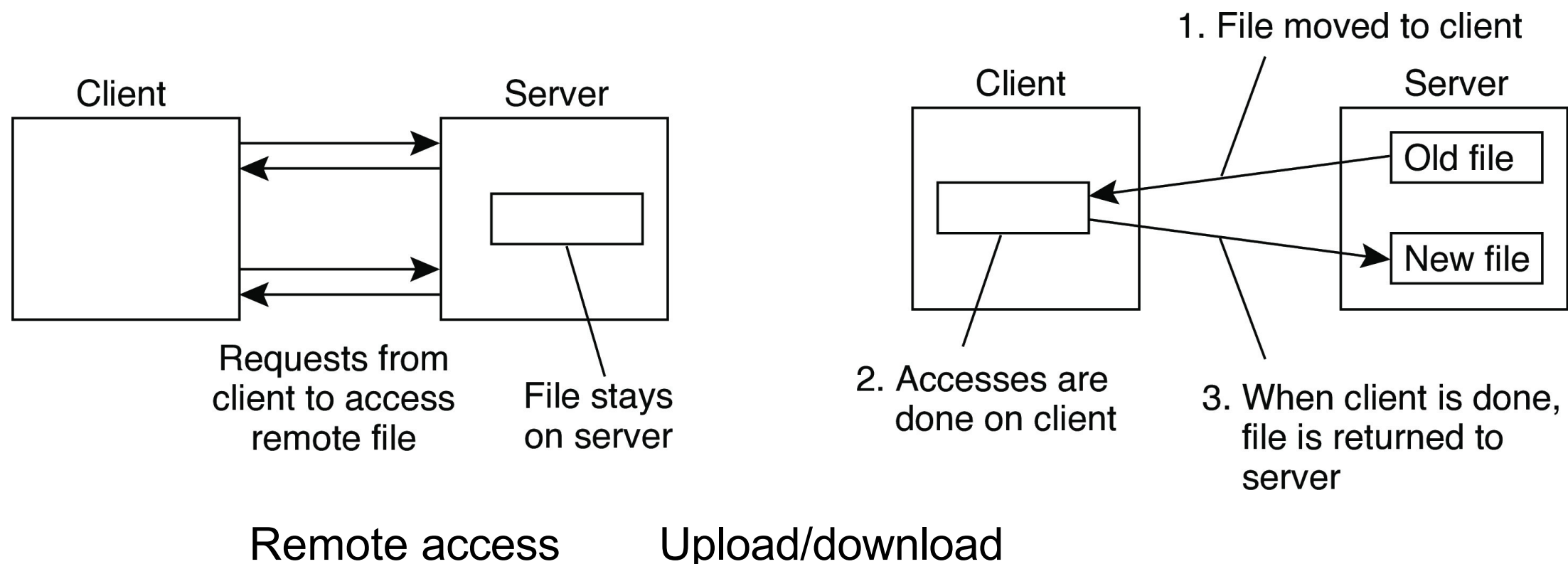
Sometimes a server also needs to act as a client. In a typical setup, programs in the processing layer usually run on a separate server. However, they can also be spread across both client and server machines. A classic example of where a three-tier architecture comes into play is transaction processing. Here, a separate process called the transaction processing monitor coordinates all transactions, potentially across different data servers. Another common example of a three-tier architecture is in the setup of websites. In this case, a web server acts as the entry point to the site and passes requests to an application server where the actual processing happens. This application server then interacts with a database server.

Layered-System

Example: The Network File System

Foundations

Each NFS server provides a standardized view of its local file system: each server supports the same model, regardless the implementation of the file system. NFS uses a communication protocol that lets clients access files on a server, enabling a diverse collection of processes, running on different operating systems and machines, to share a common file system.

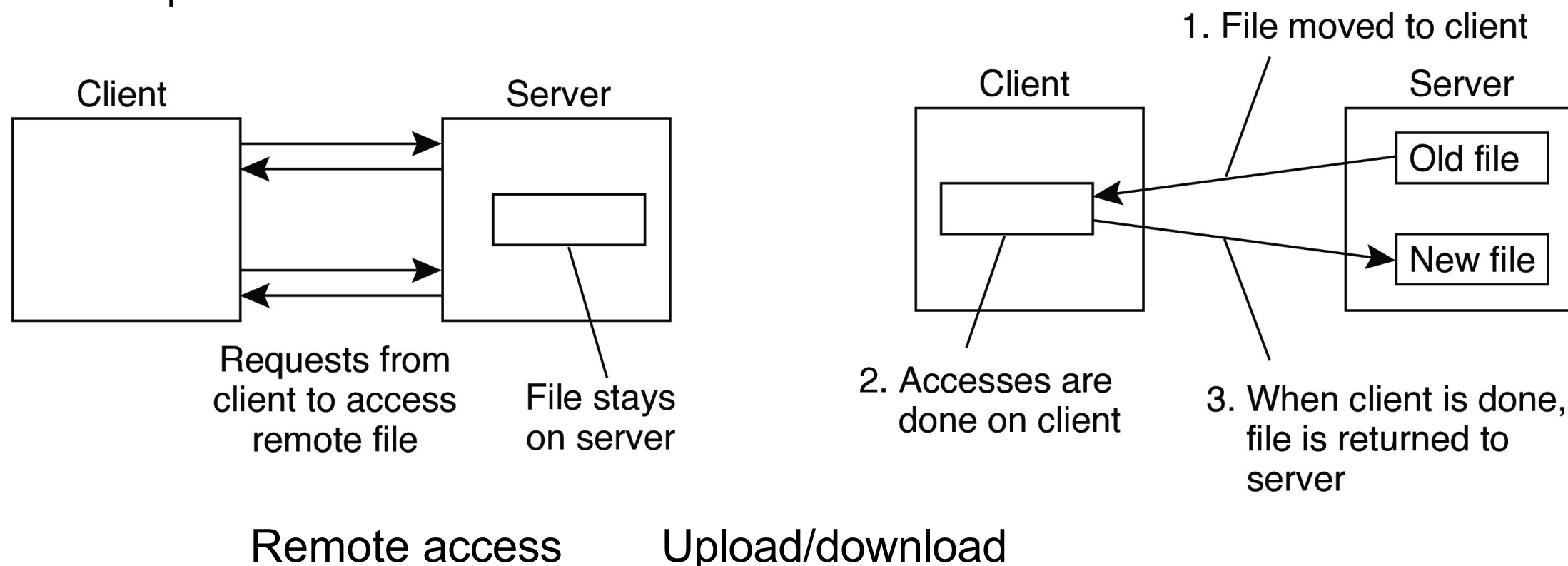


Layered-System

Example: The Network File System

The NFS remote access model

NFS and similar systems are based on the remote file service model. In this model, clients have transparent access to a file system managed by a remote server. Clients don't usually know where the files are actually located. They interact with an interface similar to a local file system interface. The client has access to various file operations, but it's the server that actually carries out these operations. The NFS remote access model

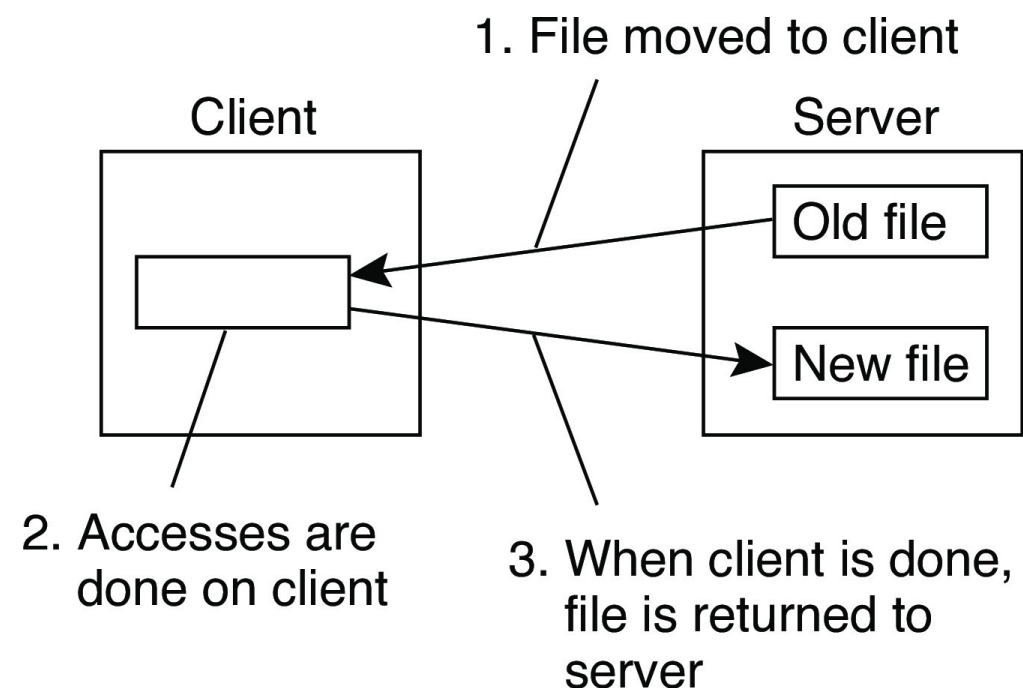
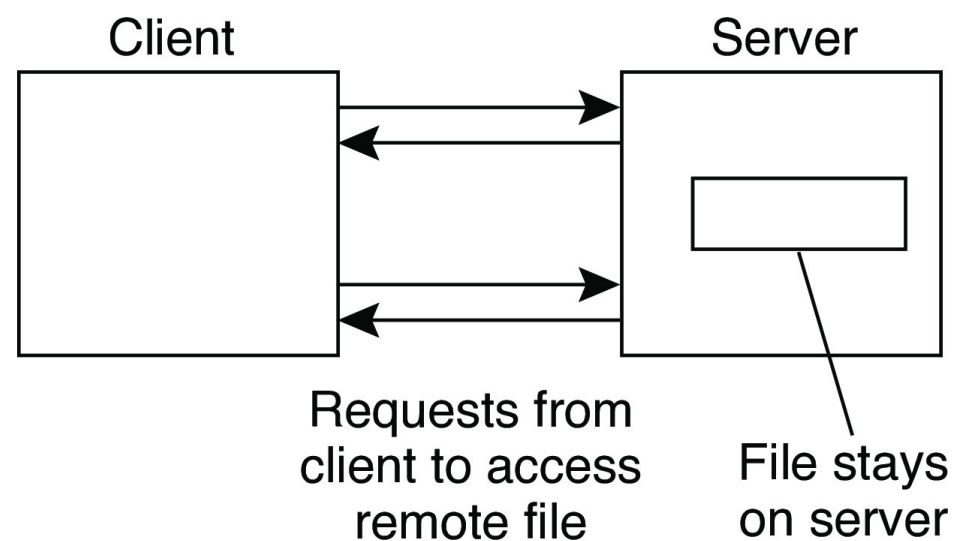


Layered-System

Example: The Network File System

Upload/download access model

In contrast, the upload/download model works differently. Here, a client downloads a file from the server, works on it locally, and then uploads it back to the server when done. This way, another client can use the file later. The Internet's FTP service works like this when a client downloads a complete file, modifies it, and then uploads it back.



Note

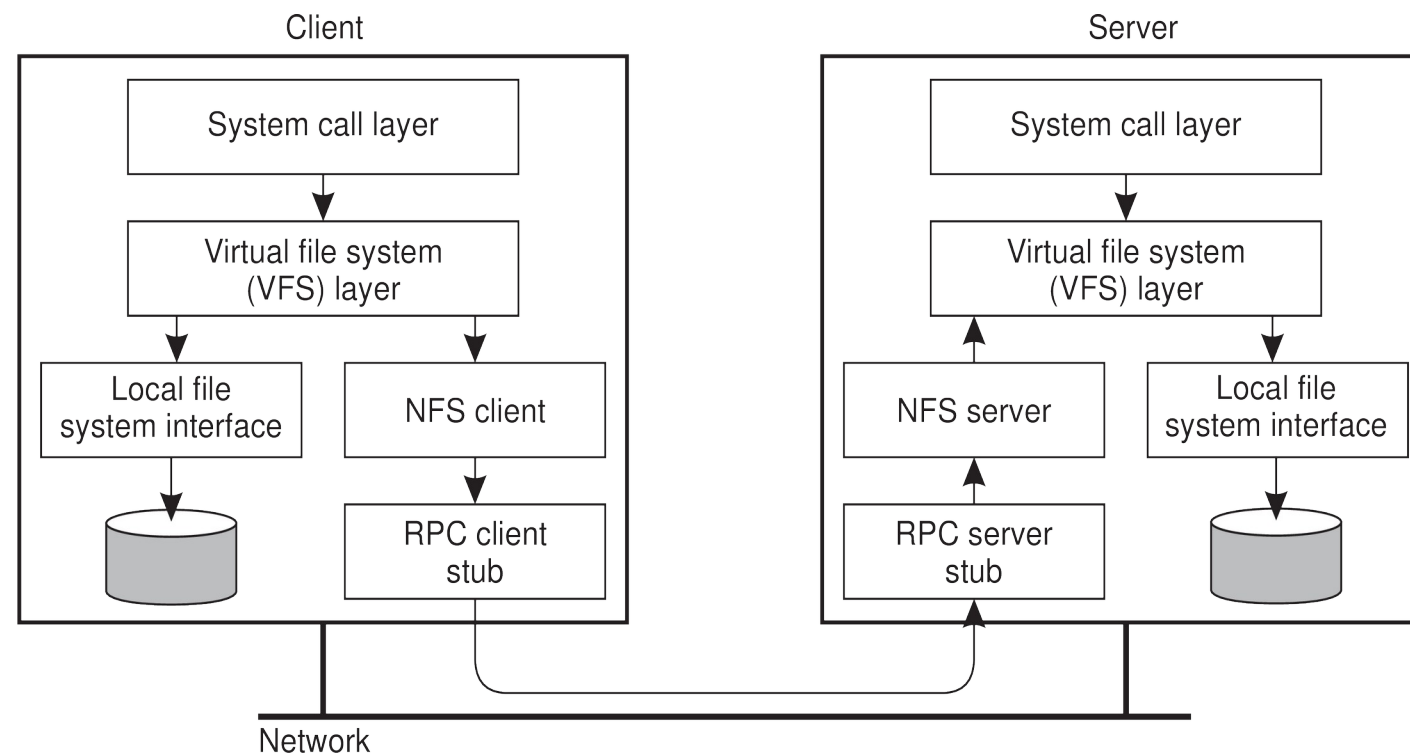
Remote access

Upload/download

FTP is a typical upload/download model. The same can be said for systems like Dropbox.

Layered-System

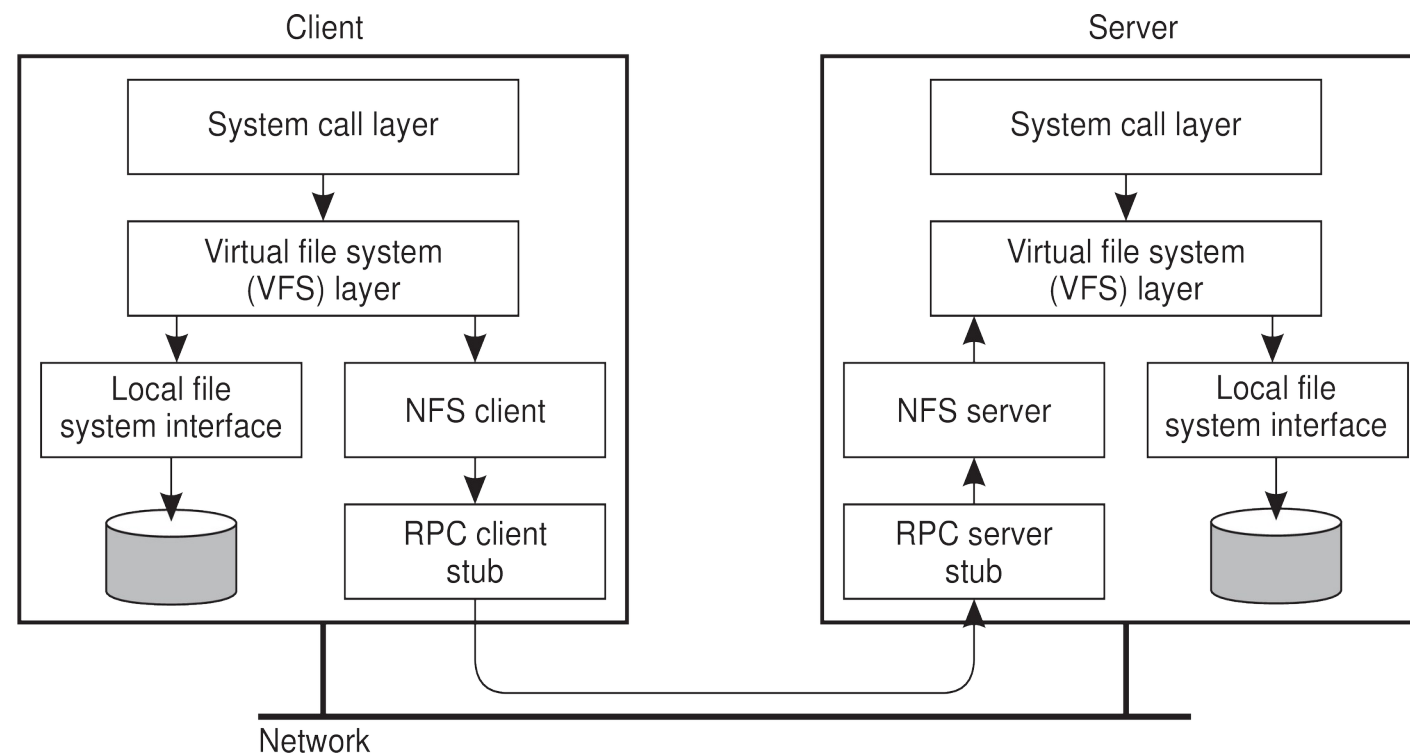
NFS architecture: Modern Unix Systems



A client accesses the file system using the system calls provided by its local operating system. However, instead of the local Unix file system interface, it uses an interface to the Virtual File System (VFS), which is now a standard for interfacing with different (distributed) file systems. Most modern operating systems support VFS. Without it, developers would have to reimplement large parts of an operating system when adopting a new file-system structure. With NFS, operations on the VFS interface are either passed to a local file system or to the NFS client, which handles access to files stored on a remote server.

Layered-System

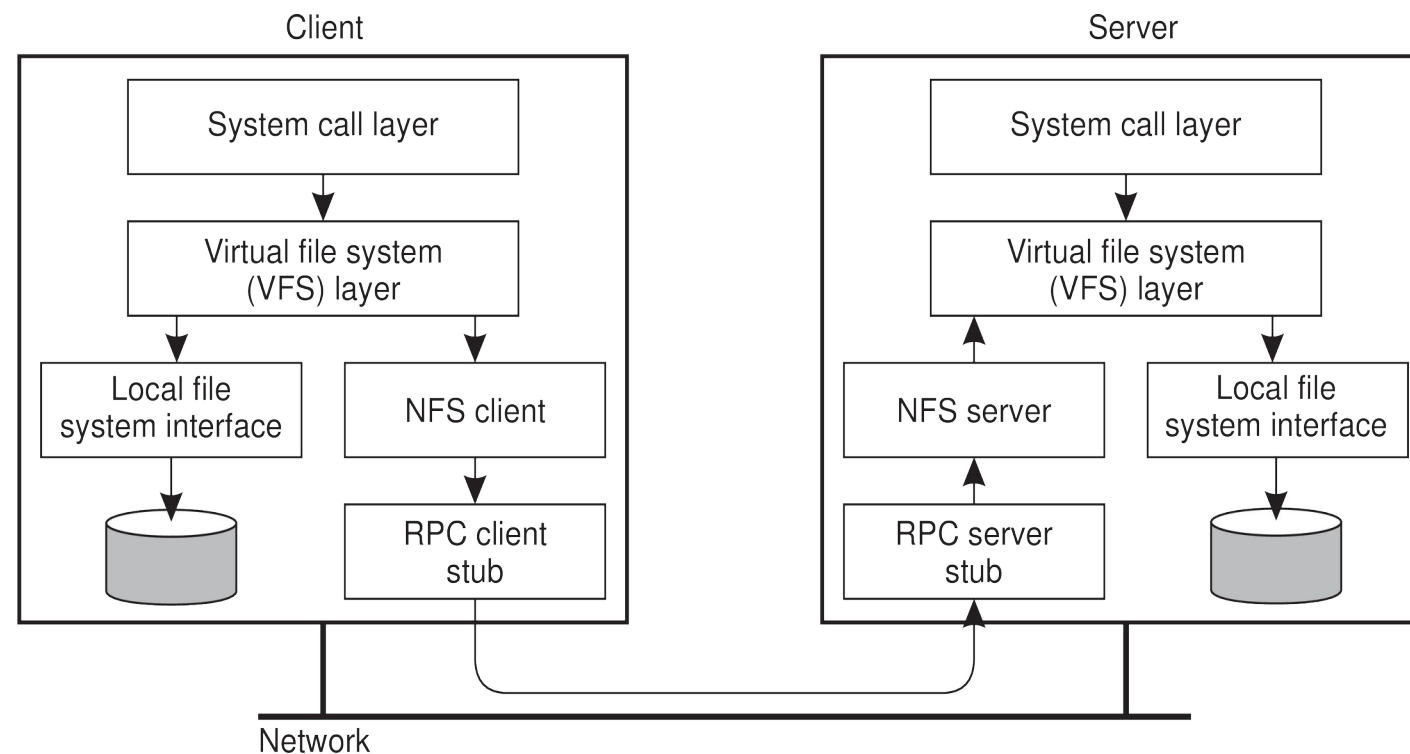
NFS architecture: Modern Unix Systems



All client-server communication in NFS is done through remote procedure calls (RPCs). An RPC allows a client on one machine to make a call to a procedure implemented on another machine. The NFS client implements the NFS file system operations as remote procedure calls to the server. The operations provided by the VFS interface can be different from those provided by the NFS client. The idea behind VFS is to hide the differences between various file systems.

Layered-System

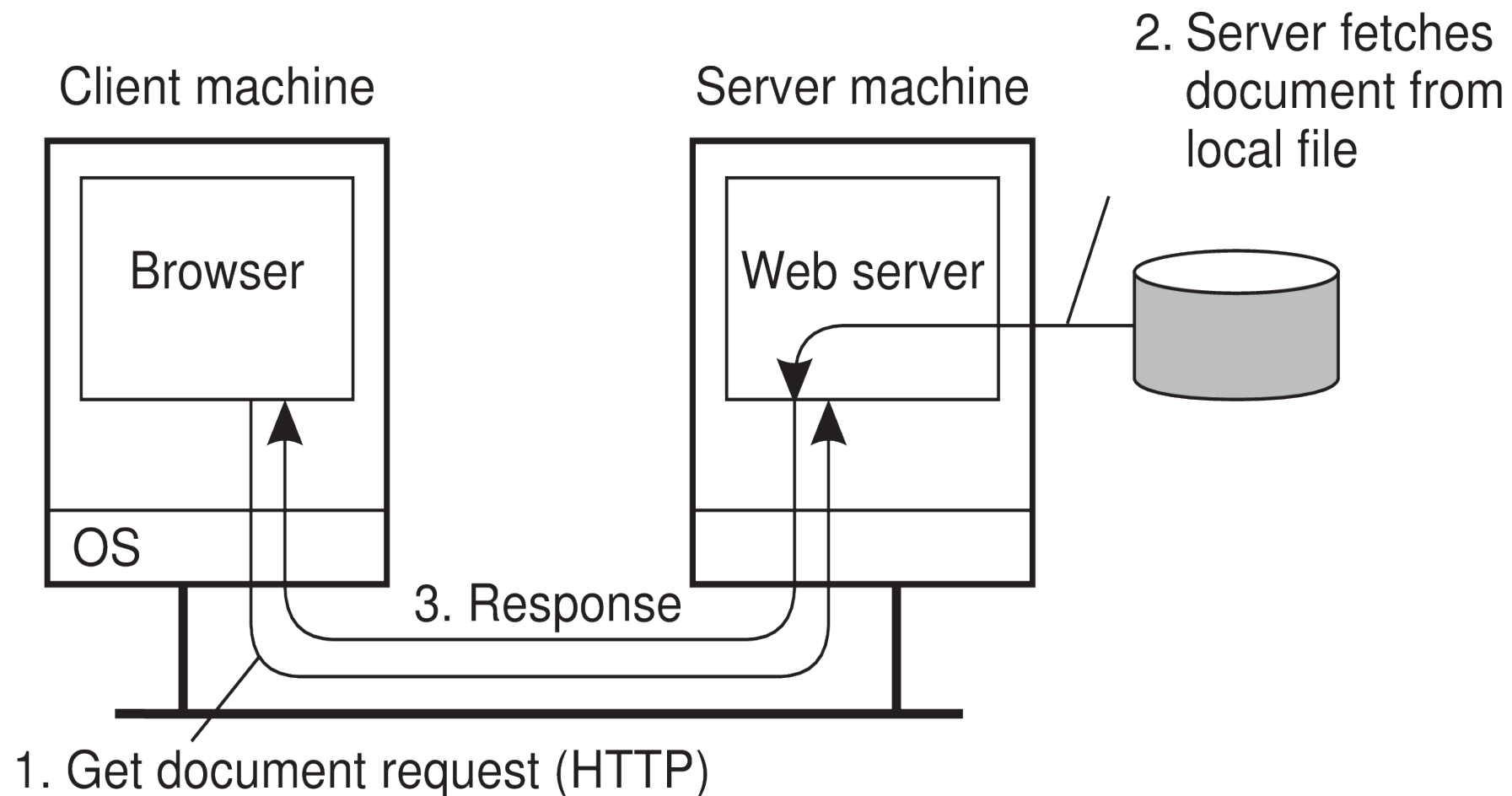
NFS architecture: Modern Unix Systems



On the server side, there's a similar setup. The NFS server handles incoming client requests. The RPC component at the server translates incoming requests into regular VFS file operations, which are then passed to the VFS layer. The VFS is responsible for implementing a local file system where the actual files are stored. One big advantage of this setup is that NFS is mostly independent of local file systems. It doesn't matter whether the client or server is using a Unix file system, or a Windows file system. The important thing is that these file systems comply with the NFS file system model.

Layered-System

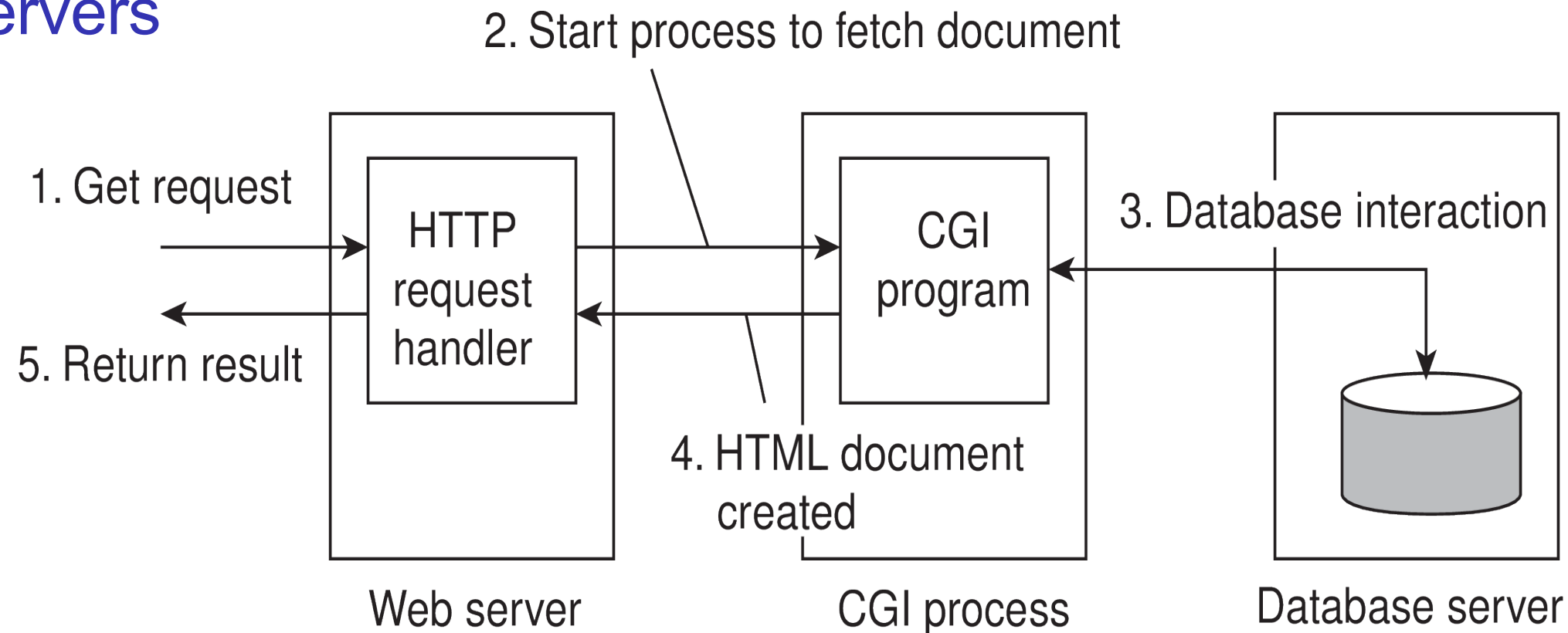
Example: Simple Web servers



- A website consisted as a collection of HTML files
- HTML files could be referred to each other by a hyperlink
- A Web server essentially needed only a hyperlink to fetch a file
- A browser took care of properly rendering the content of a file

Layered-System

Example: Less simple Web servers



- A website was built around a database with content
- A Webpage could still be referred to by a hyperlink
- A Web server essentially needed only a hyperlink to fetch a file
- A separate program (Common Gateway Interface) composed a page
- A browser took care of properly rendering the content of a file

Alternative Organizations

Vertical distribution

Comes from dividing distributed applications into three logical layers, and running the components from each layer on a different server (machine).

Horizontal distribution

A client or server may be physically split up into logically equivalent parts, but each part is operating on its own share of the complete data set.

Peer-to-peer architectures

Processes are all equal: the functions that need to be carried out are represented by every process $\Rightarrow$ each process will act as a client and a server at the same time (i.e., acting as a **servant**).

Structured P2P Systems

Essence

Make use of a **semantic-free index**: each data item is uniquely associated with a key, in turn used as an index. Common practice: use a **hash function**

key(data item) = hash(data item's value).

P2P system now responsible for storing *(key,value)* pairs.

Distributed Hash Table

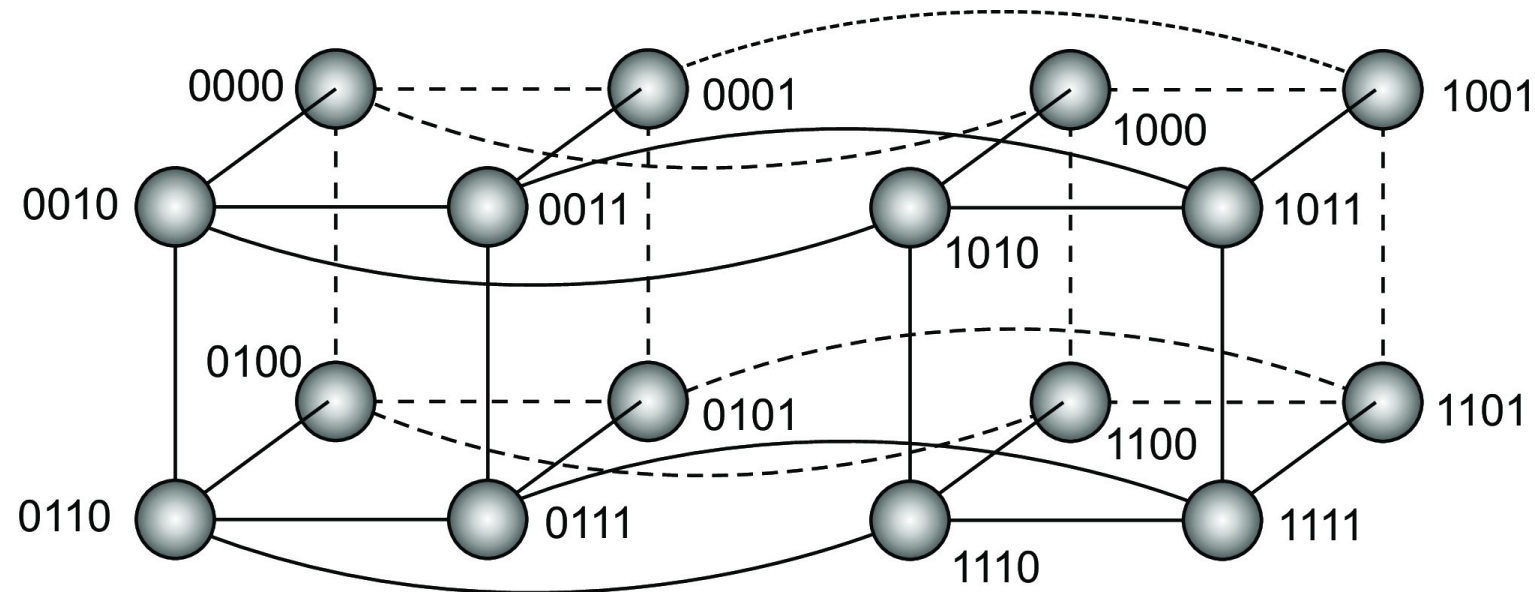
The system stores *(key, value)* pairs, with each node assigned an identifier from the set of all possible hash values. Each node is responsible for storing data associated with a specific subset of keys.

The main job of a structured peer-to-peer system is to look up data items by their keys. The system efficiently maps a key to the node that stores the corresponding data.

The structured topology of the system is crucial here. Any node can be asked to look up a key, and it routes the request to the node responsible for that key.

Structured P2P Systems

Simple example: hypercube: A hypercube is an n-dimensional cube



Looking up d with **key** $k \in \{0, 1, 2, \dots, 2^4 - 1\}$ means **routing** request to node with **identifier** k .

A four-dimensional hypercube looks like two ordinary cubes, each with 8 vertices and 12 edges, connected together.

In this system, each data item is associated with one of the 16 nodes. This is done by hashing the data item's value to a key in the range $\{0, 1, 2, \dots, 15\}$. Suppose the node with identifier 0111 needs to look up the data with key 14, which in binary is 1110. The node 1110 is responsible for storing data with key 14. Node 0111 can forward the request to a neighbor closer to 1110, like node 0110 or node 1111. If it picks node 0110, that node will then forward the request directly to node 1110, where the data can be retrieved.

Structured P2P Systems

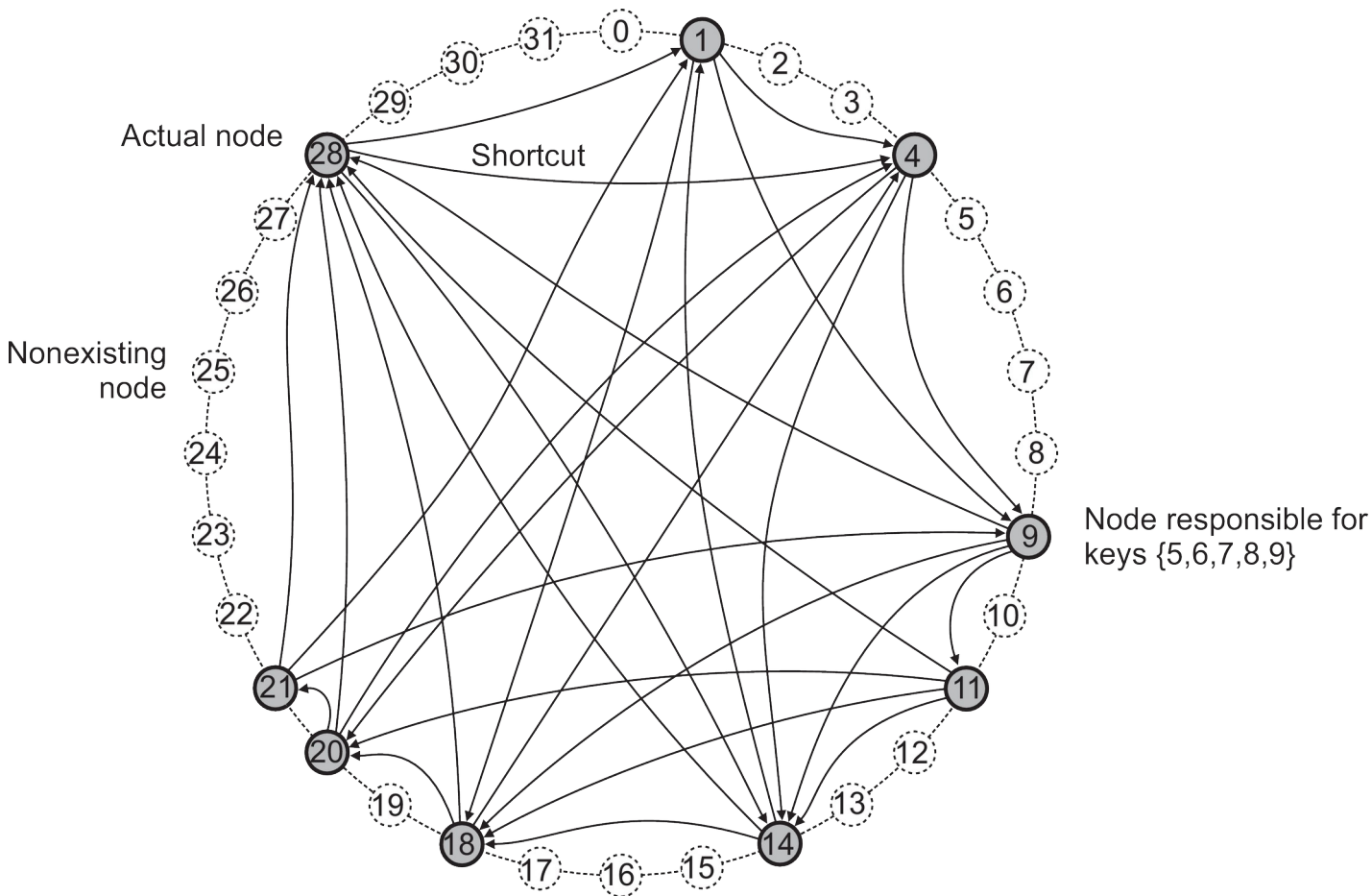
Example: Chord

Principle

- Nodes are logically organized in a ring. Each node has an m -bit **identifier**.
- Each data item is hashed to an m -bit **key**.
- Data item with key k is stored at node with smallest identifier $id \geq k$, called the **successor** of key k .
- The ring is extended with various **shortcut links** to other nodes.
- Typically, keys and identifiers are 128 or 160 bits long.

Structured P2P Systems

Example: Chord



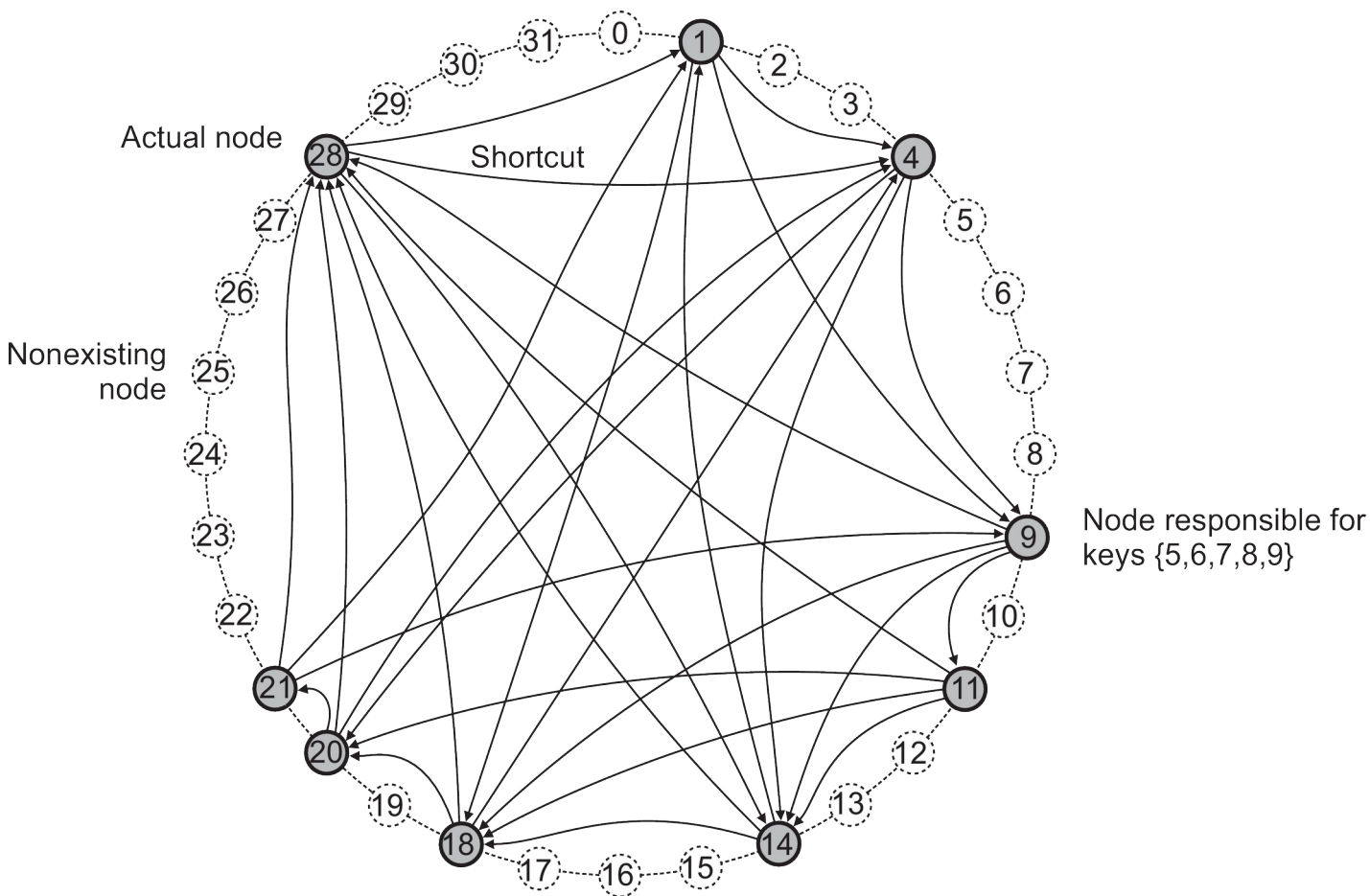
lookup(3)@9 : 28 → 1 → 4

Imagine a smaller Chord ring with $m = 5$ bits and nine nodes $\{1, 4, 9, 11, 14, 18, 20, 21, 28\}$. The successor of key 7 is node 9. Similarly, $\text{succ}(5) = 9$, and $\text{succ}(9) = 9$. In Chord, each node has shortcuts to other nodes, represented by directed edges.

To look up a key, a node forwards the request as far as possible without passing the node responsible for that key. For example, in our Chord ring, if node 9 is asked to look up the node responsible for key 3 (which is node 4), it uses its shortcuts to other nodes: 11, 14, 18, and 28. Node 28 is the farthest node 9 knows about that precedes the node responsible for key 3, so it forwards the request to node 28. Node 28 has shortcuts to nodes 1, 4, and 14, but doesn't know about nodes between 1 and 4. So, it forwards the request to node 1, which knows its successor is node 4, responsible for key 3.

Structured P2P Systems

Example: Chord



Suppose a node with a unique identifier u wants to join a Chord overlay. It contacts any node and asks it to look up u , returning the value $v = \text{succ}(u)$. Node u then inserts itself between the predecessor of v and v itself, becoming the new predecessor of v . During this process, shortcuts from u to other nodes are established, and some shortcuts previously pointing to v are updated to point to u . Any data item with key k stored at v , but for which $\text{succ}(k)$ is now u , is transferred from v to u . When a node wants to leave, it informs its predecessor and successor and transfers its data items to $\text{succ}(u)$.

$\text{lookup}(3)@9 : 28 \rightarrow 1 \rightarrow 4$

Unstructured P2P Systems

Essence

Each node maintains an ad hoc list of neighbors. The resulting overlay resembles a **random graph**: an edge $\langle u, v \rangle$ exists only with a certain probability $P[\langle u, v \rangle]$.

Searching

- **Flooding**: issuing node u passes request for d to all neighbors. Request is ignored when receiving node had seen it before. Otherwise, v searches locally for d (recursively). May be limited by a **Time-To-Live**: a maximum number of hops.
- **Random walk**: issuing node u passes request for d to randomly chosen neighbor, v . If v does not have d , it forwards request to one of *its* randomly chosen neighbors, and so on.

Unstructured P2P Systems

- **Model:** Assume N nodes and that each data item is replicated across r randomly chosen nodes.
- **Random Walk**

$P[k]$ probability that item is found after k attempts:

$$\mathbb{P}[k] = \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1}$$

S (“search size”) is expected number of nodes that need to be probed:

$$S = \sum_{k=1}^N k \cdot \mathbb{P}[k] = \sum_{k=1}^N k \cdot \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1} \approx N/r \text{ for } 1 \ll r \leq N.$$

Unstructured P2P Systems

- Flooding:

- Flood to d randomly chosen neighbors
- After k steps, some $R(k) = d \cdot (d - 1)^k - 1$ will have been reached (assuming k is small).
- With fraction r/N nodes having data, if $r/N \cdot R(k) \geq 1$, we will have found the data item.

- Comparison

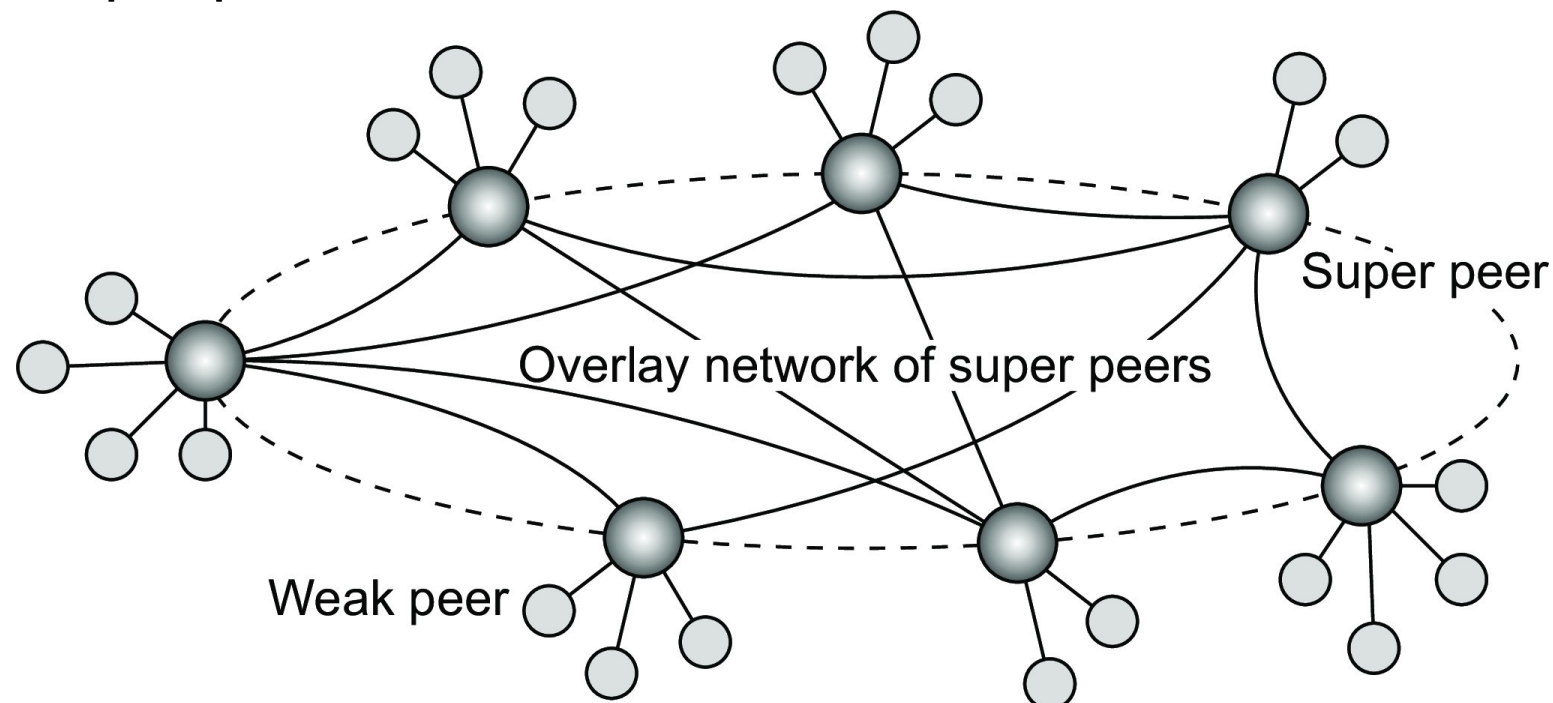
- If $r/N = 0.001$, then $S \approx 1000$
- With flooding and $d = 10$, $k = 4$, we contact 7290 nodes.
- Random walks are more communication efficient, but might take longer before they find the result.

Hierarchically Organized P2P Networks

Essence

It is sometimes sensible to break the symmetry in pure peer-to-peer networks:

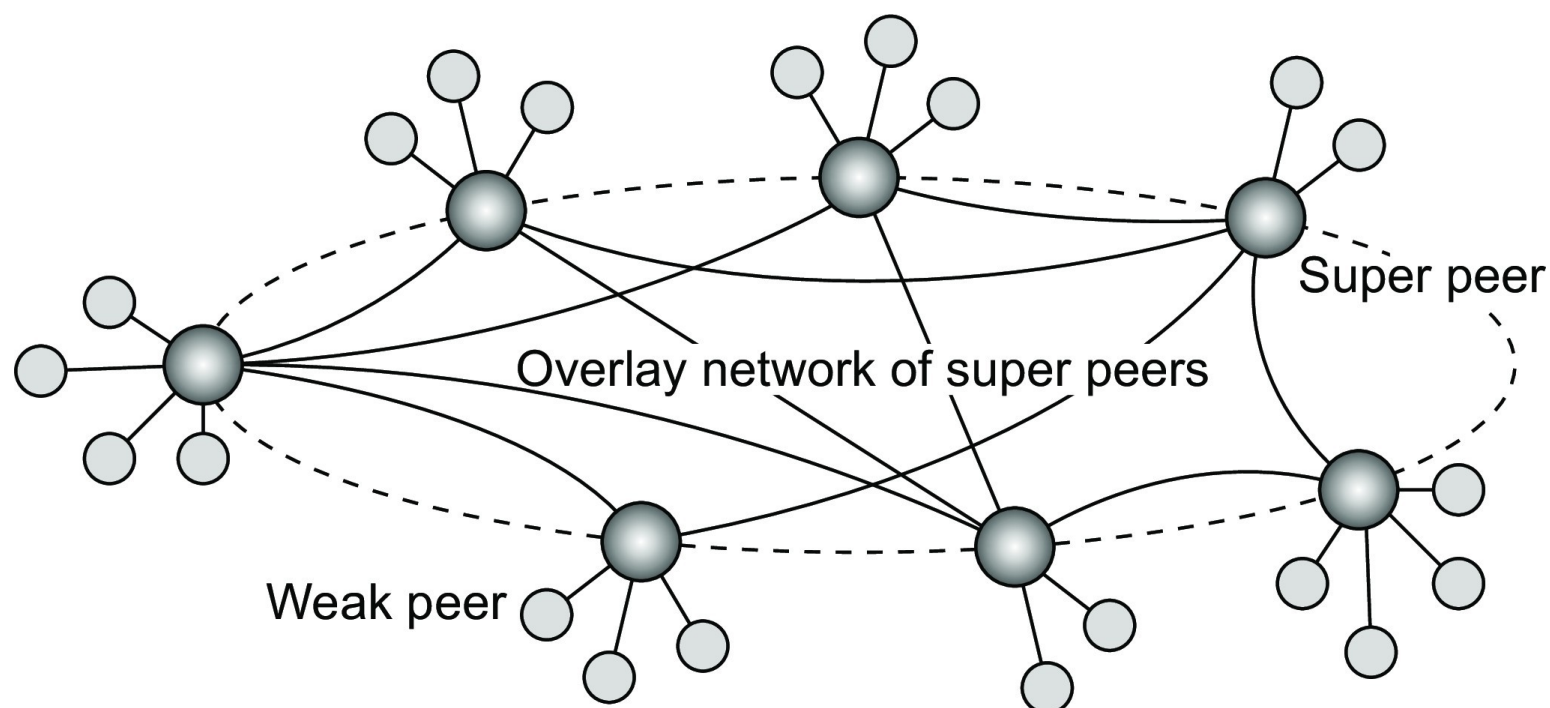
- When searching in unstructured P2P systems, having **index servers** improves performance
- Deciding where to store data can often be done more efficiently through **brokers**. A broker that collects data on resource usage and availability can quickly find a node with enough resources.
- These special nodes that maintain an index or act as brokers are called **super peers**. Super peers are usually organized in their own peer-to-peer network, creating a hierarchical structure. In this setup, every regular peer, now called a **weak peer**, connects to a super peer. All communication for the weak peer goes through its super peer.



Hierarchically Organized P2P Networks

Essence

- Typically, when a weak peer joins the network, it attaches to one super peer and stays connected to it until it leaves the network. Super peers are expected to be stable and highly available. To handle potential instability, backup schemes are used, like pairing each super peer with another and having weak peers connect to both.
- However, fixed associations with super peers might not always be ideal. In file-sharing networks, for instance, it might be better for a weak peer to connect to a super peer that has an index of files the weak peer is interested in. This increases the chances that the super peer will know where to find the file. There is a scheme where weak peers can change their super peer as they find better ones. A super peer that successfully helps with a lookup operation gets preference over others.

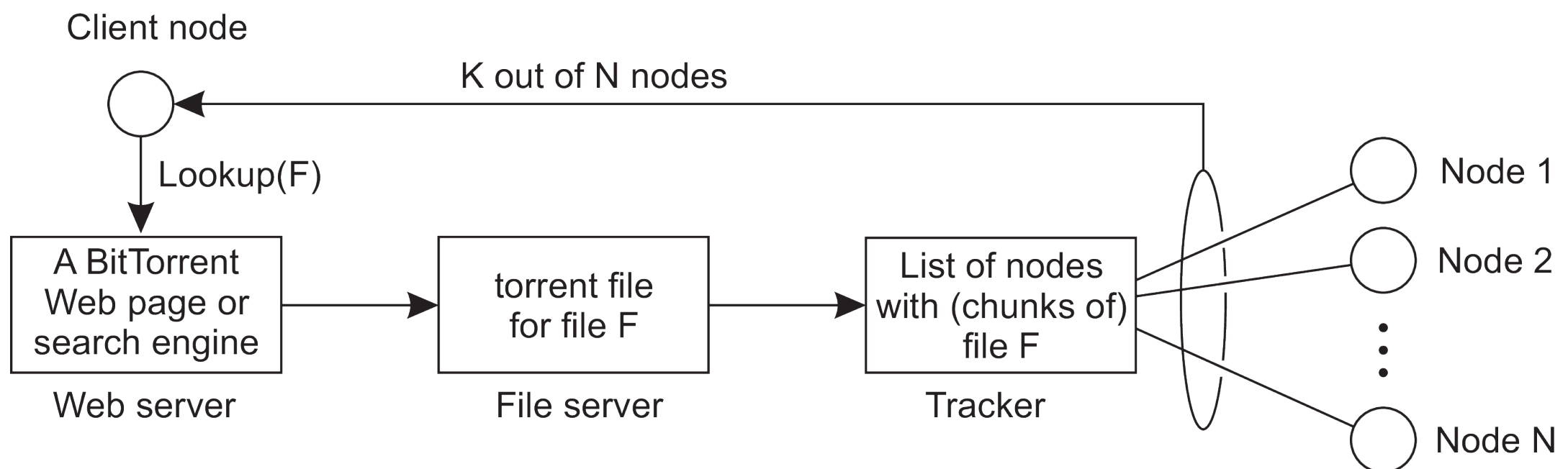


Hierarchically Organized P2P Networks

Example: BitTorrent

Principle: search for a file F

- Lookup file at a global directory $\Rightarrow$ returns a **torrent file**
- Torrent file contains reference to **tracker**: a server keeping an accurate account of **active** nodes that have (chunks of) F .
- P can join **swarm**, get a chunk for free, and then trade a copy of that chunk for another one with a peer Q also in the swarm.

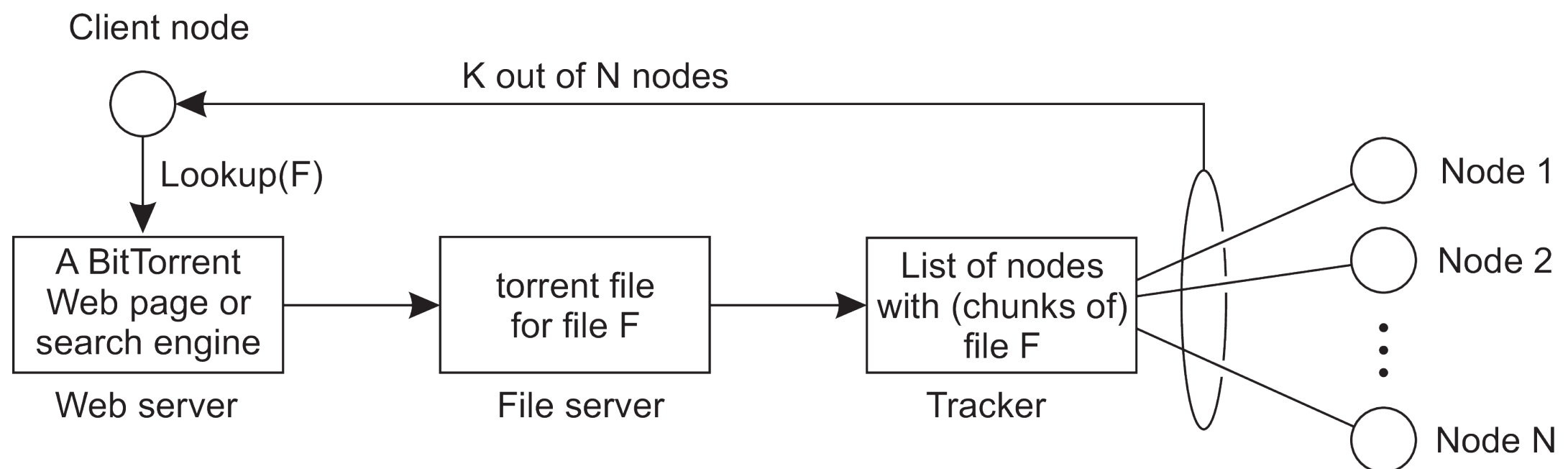


Hierarchically Organized P2P Networks

Example: BitTorrent

How BitTorrent Works

- The main idea is that when you want a file, you download chunks of it from other users. Once you have all the chunks, they are assembled into the complete file. BitTorrent was designed to encourage collaboration. In most file-sharing systems, a lot of users just download files and don't contribute anything back. To prevent this, BitTorrent only allows you to download a file if you're also uploading content to someone else.

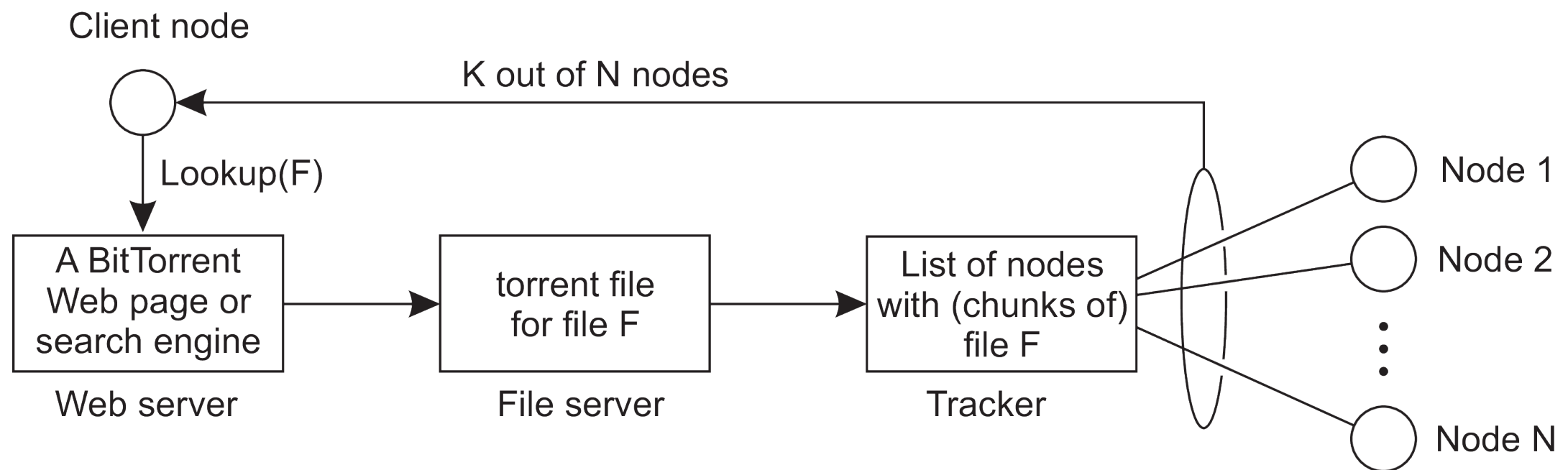


Hierarchically Organized P2P Networks

Example: BitTorrent

Getting Started with BitTorrent

- To download a file, you first go to a global directory, usually found on a well-known website. This directory has references to torrent files, which contain the information needed to download the specific file. A torrent file includes a link to a tracker—a server that keeps track of which nodes (users) have chunks of the requested file. These active nodes are also downloading the file. There are many trackers, but usually only one tracker per file or set of files.

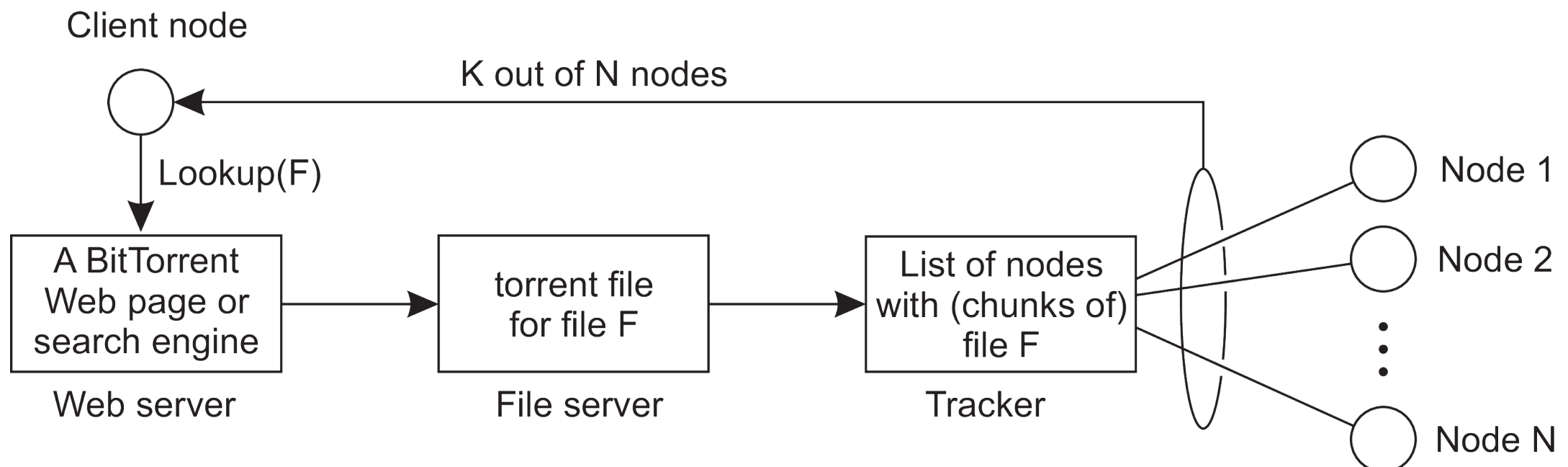


Hierarchically Organized P2P Networks

Example: BitTorrent

Downloading and Sharing

- Once you find the nodes with the chunks you need, you become an active node yourself. At this point, you must help others by providing chunks of the file that they don't have yet. There's a simple rule to enforce this: if Node P notices that Node Q is downloading more than it's uploading, P can slow down the rate at which it sends data to Q. This system works well as long as P has something to download from Q. That's why nodes often get references to many other nodes, giving them better opportunities to trade data.

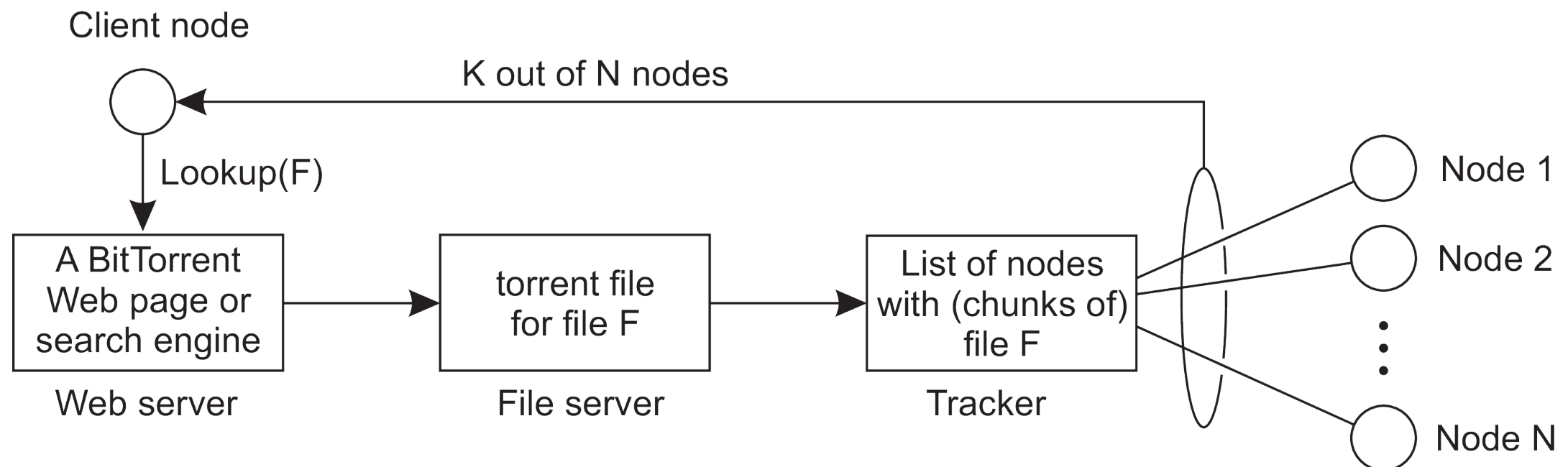


Hierarchically Organized P2P Networks

Example: BitTorrent

Centralized and Decentralized Solutions

- BitTorrent combines centralized and decentralized approaches. The system's bottleneck is often the trackers. To address this, some implementations of BitTorrent have nodes join a separate structured peer-to-peer system to help with tracking file downloads. This spreads the tracker's load across the participating nodes, with each node acting as a tracker for a small set of torrent files. The original tracker still coordinates the file download initially, but in many modern BitTorrent systems, the tracking function is minimized to just providing an initial list of peers. After that, the new peer communicates directly with those peers and not with the tracker anymore.



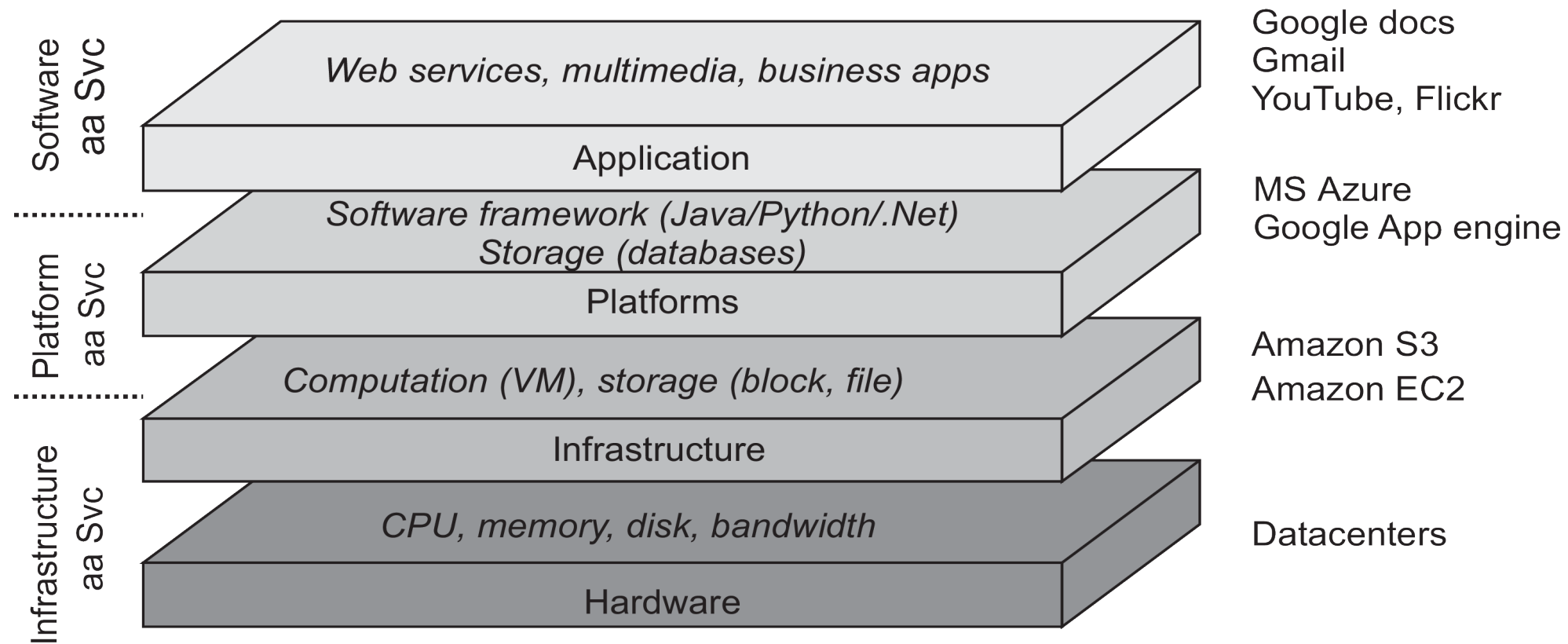
Hybrid System: Cloud Computing

Essence

Real-world distributed systems are pretty complex because they mix various architectures. You'll find centralized features combined with peer-to-peer setups and hierarchical organizations all in one system. The complexity increases because many of these systems span across different organizations, leading to decentralized solutions where no single organization is in charge of the whole system's operation. Now, we will take a closer look into these complex, hybrid system architectures to understand how they work.

Organizations running data centers have been looking for ways to open up their resources to customers. This eventually led to the idea of utility computing, where customers could upload tasks to a data center and pay based on the resources they used.

Hybrid System: Cloud Computing



Cloud Computing

Cloud computing is all about having an easy-to-use and accessible pool of virtualized resources. You can configure which resources to use and how to use them dynamically, which is great for scalability. If you need more resources, you can just get more. Cloud computing follows a pay-per-use model, often with customized service-level agreements (SLAs) to guarantee certain performance levels. Clouds are generally organized into four layers.

Hybrid System: Cloud Computing

Make a distinction between four layers

- **Hardware**: Processors, routers, power and cooling systems. Customers normally never get to see these.
- **Infrastructure**: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.
- **Platform**: Provides higher-level abstractions for storage and such. Example: Amazon S3 storage system offers an API for (locally created) files to be organized and stored in so-called **buckets**.
- **Application**: Actual applications, such as office suites (text processors, spreadsheet applications, presentation applications). Comparable to the suite of apps shipped with OSes.

Hybrid System: Cloud Computing

Services

- **Infrastructure-as-a-Service (IaaS)**: covering the hardware and infrastructure layer
- **Platform-as-a-Service (PaaS)**: covering the platform layer.
- **Software-as-a-Service (SaaS)**: in which their applications are covered.

Hybrid System: Edge-cloud architecture

Essence

Systems deployed on the Internet where servers are placed **at the edge** of the network: the boundary between enterprise networks and the actual Internet.

What is Edge Computing

Edge computing deals with placing services "at the edge" of the network. This edge is typically the boundary between enterprise networks and the actual Internet, often provided by an Internet Service Provider (ISP). For instance, many universities have campuses with various buildings, each with its own local network, all connected through a campus-wide network. These campuses may have multiple on-premise services for storage, computing, security, lectures, and so on. On-premise means the local IT department hosts these services on servers directly connected to the campus network. Much of the traffic for these services stays within the campus network, forming a typical edge infrastructure.

Hybrid System: Edge-cloud architecture

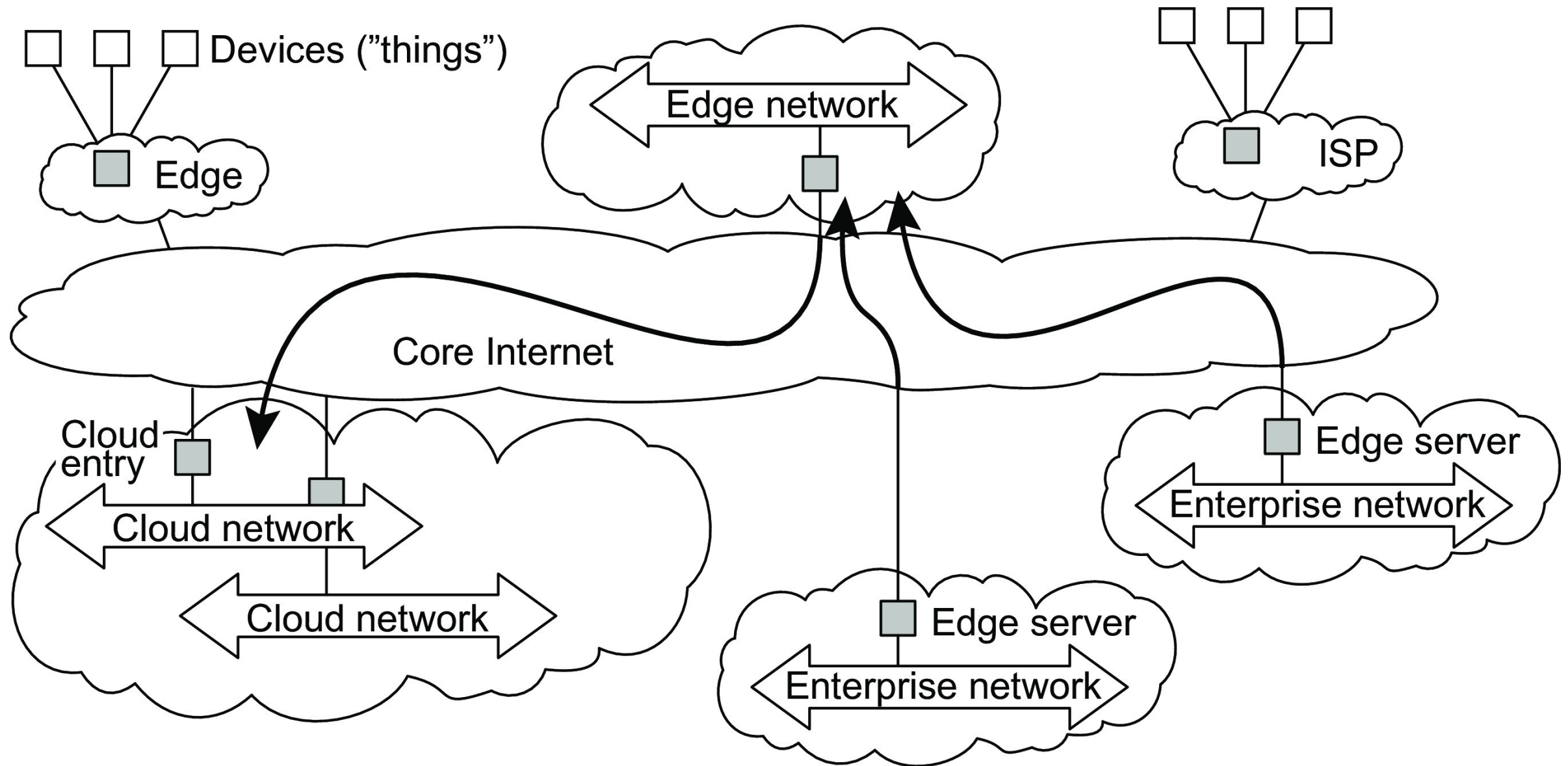
Connecting Beyond the Campus

These on-premise servers can also connect to servers at other universities or use additional servers elsewhere. Instead of setting up peer-to-peer connections between universities, we often see configurations where various universities share services through a logically centralized infrastructure. This infrastructure might be in the cloud or set up through regional data centers. As we get closer to cloud infrastructures, we use the term fog computing.

Why Do We Need Edge Computing?

Many configurations for edge infrastructure come to mind, such as tracking activities, layered video-streaming infrastructures, gaming infrastructures, etc. What they all have in common is that smart end devices need to (eventually) connect to a service hosted somewhere in the cloud. So, why do we need edge infrastructure at all? Logically, it seems simpler to connect directly to the cloud service using existing networking facilities. Let's take a closer look at some arguments for edge computing.

Hybrid System: Edge-cloud architecture



Hybrid System: Edge-cloud architecture

Reasons for having an edge infrastructure

Commonly (and often misconceived) arguments

- **Latency and bandwidth:** Especially important for certain real-time applications, such as augmented/virtual reality applications. Many people underestimate the latency and bandwidth to the cloud.
- **Reliability:** The connection to the cloud is often assumed to be unreliable, which is often a false assumption. There may be critical situations in which extremely high connectivity guarantees are needed.
- **Security and privacy:** The implicit assumption is often that when assets are nearby, they can be made better protected. Practice shows that this assumption is generally false. However, securely handling data operations in the cloud may be trickier than within your own organization.

Hybrid System: Edge-cloud architecture

Edge orchestration

Managing resources at the edge may be trickier than in the cloud

- **Resource allocation**: we need to guarantee the availability of the resources required to perform a service.
- **Service placement**: we need to decide **when** and **where** to place a service. This is notably relevant for mobile applications.
- **Edge selection**: we need to decide which edge infrastructure should be used when a service needs to be offered. The closest one may not be the best one.

Observation

There is still a lot of buzz about edge infrastructures and computing, yet whether all that buzz makes any sense remains to be seen.

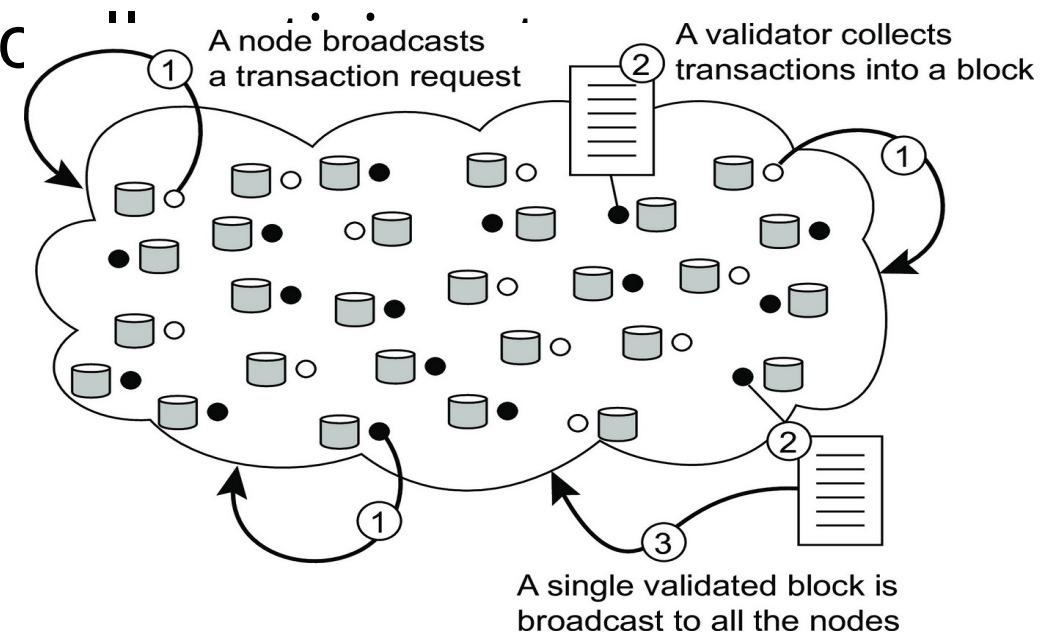
Hybrid System: Blockchains

- Principle working of a blockchain system
- Blockchain systems enable the registration of transactions.
- The key issue with any transaction system is making sure a transaction is checked, completed, and stored for future reference. For instance, if Alice wants to send \$10 to Bob, she'd normally go to the bank. She'd sign something to show she really wants to do it. It doesn't matter if this is done physically or digitally. The bank then checks if Alice has enough money and if Bob can receive it. If everything is good, the bank transfers the money and keeps a record for audits.
- In blockchain systems, transactions aren't just limited to money. They can be used for all sorts of things, like registering ID documents, tracking resource usage, electronic voting, and even sharing health records. So, it's a pretty broad concept.
- The bank operates as a trusted third party. An important design assumption for blockchains is that participating parties can, in principle, not be trusted.
- In the case of blockchains, we assume there is a (potentially very large) set of participants who jointly register transactions among them in a publicly available ledger. In this way, any participant can see what has happened and also verify the validity of a transaction. For example, in a blockchain system for digital coins, each having a unique and unforgeable ID, any participant should be able to check whether a coin has already been spent by checking all transactions that have taken place since the beginning.

Hybrid System: Blockchains

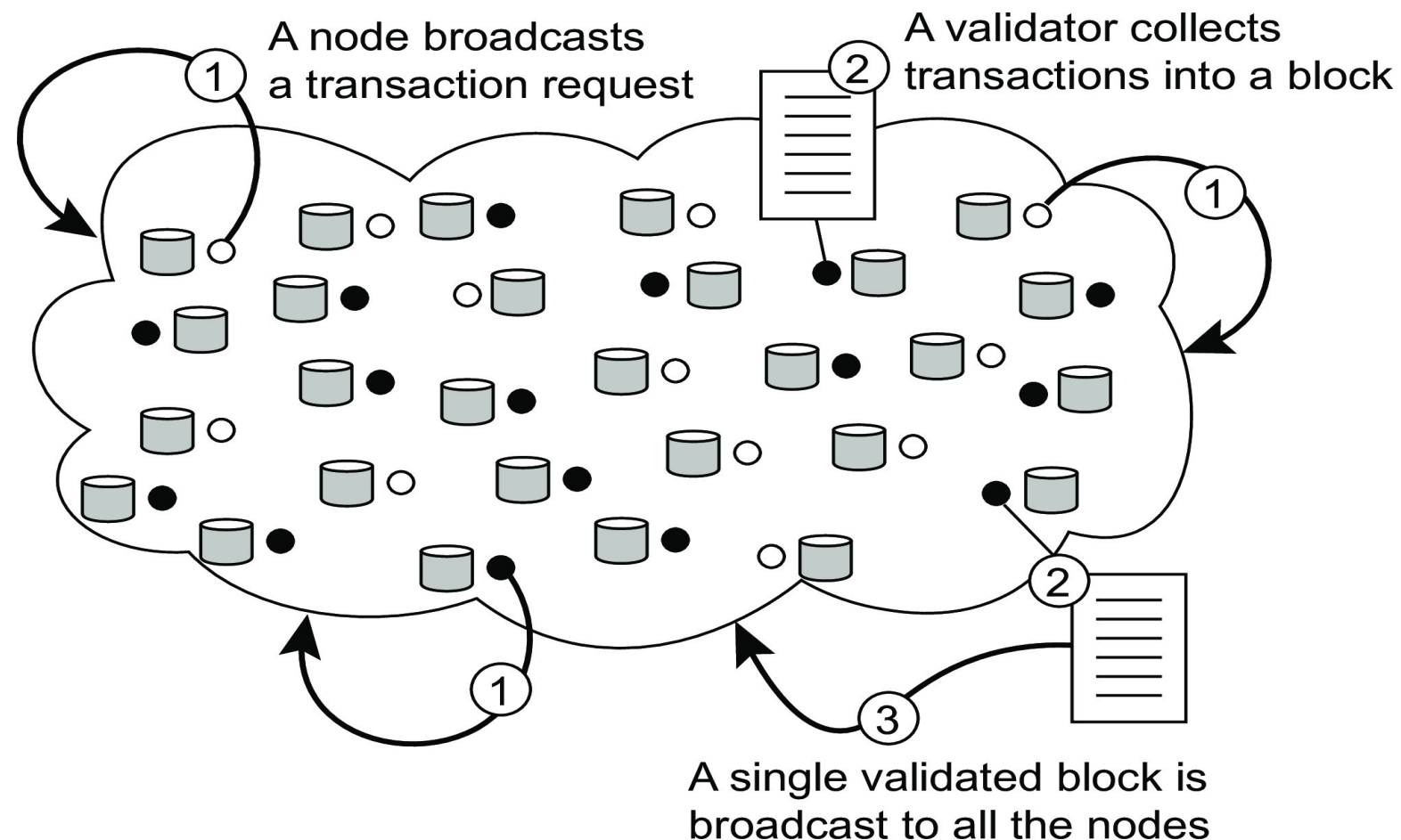
- Principle working of a blockchain system

- Step1: When Alice wants to transfer \$10 to Bob, she essentially tells all the participants in the blockchain system about this intent, thus allowing volunteers to validate the intended transaction.
- Step2: To avoid having to check every transaction separately one-by-one as they are submitted, a validator groups several transactions into a block to increase efficiency.
- Step3: If everything goes well, i.e., the transactions in the block are considered to be valid, the validator securely protects the block against any modifications, and appends the now immutable block to the chain of other blocks with validated transactions. It does so by broadcasting the validated block to



Hybrid System: Blockchains

● Principle working of a blockchain system



Observations

- Blocks are organized into an unforgeable **append-only** chain
- Each block in the blockchain is **immutable** $\Rightarrow$ massive replication
- The real snag lies in who is allowed to append a block to a chain

Hybrid System: Blockchains

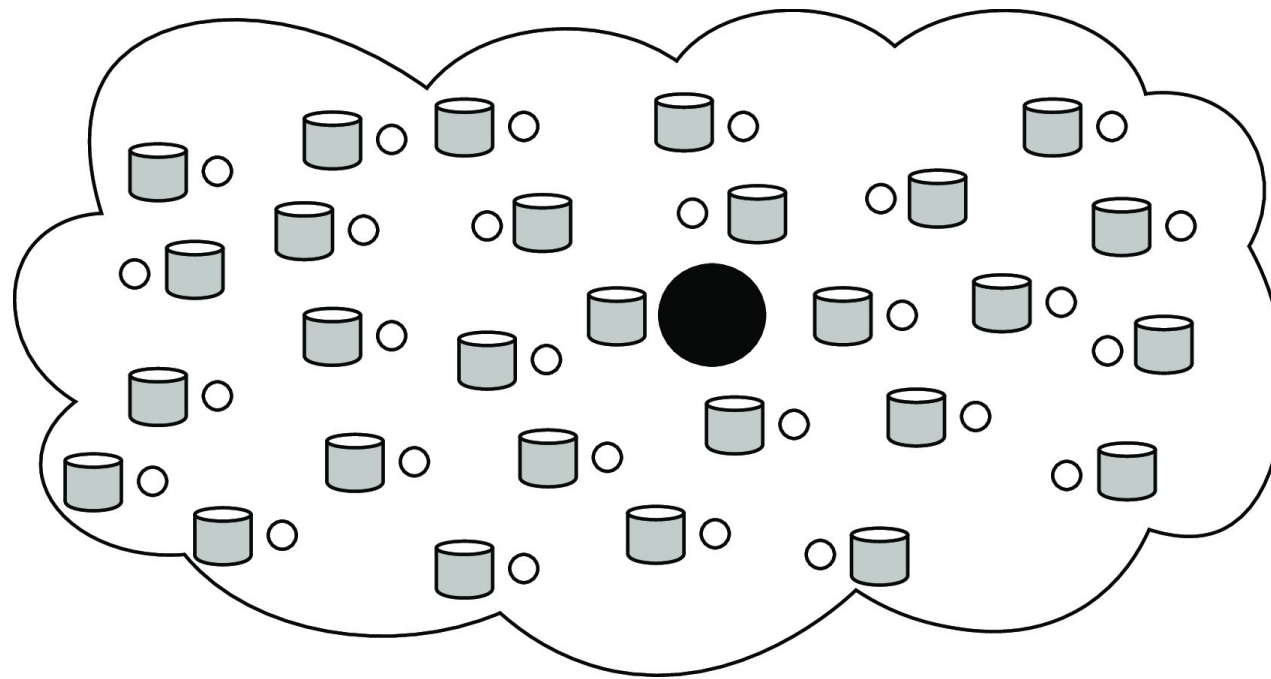
- Important notes about Blackchains
 - There's only one main chain of blocks with validated transactions in a blockchain. Each block is unchangeable, meaning if someone tries to tamper with any transaction in a block, that change will be detected.
 - Blocks can't be changed, they're perfect for being copied widely. Since they'll never be altered, it's useful for people to store copies locally to make verification easier. Basically, this single chain of unchangeable blocks gets massively replicated across the internet among all participants. This is what happens: each block is broadcast to every node in the blockchain network.
 - What sets different blockchain systems apart is how they decide which nodes get to validate transactions. In other words, we need a way to choose who gets to add a new block of validated transactions to the chain. Adding a block means everyone agrees on the transactions in it, so it's crucial to reach a consensus on which validator gets to proceed. If we don't, other validators might have to redo their work because their transactions could be affected by the new ones added to the chain.

Hybrid System: Blockchains

- Different organizations of blockchains
 - A centralized solution, in which a trusted third party validates transactions as before.
 - A distributed solution, in which a small, preselected group of processes takes over the role of a trusted third party.
 - A fully decentralized solution, in which, in principle, all participating nodes in the blockchain jointly reach consensus without any (distributed) third party. Each node participating in the blockchain is assumed to have a full copy locally available.

Hybrid System: Blockchains

Centralized solution

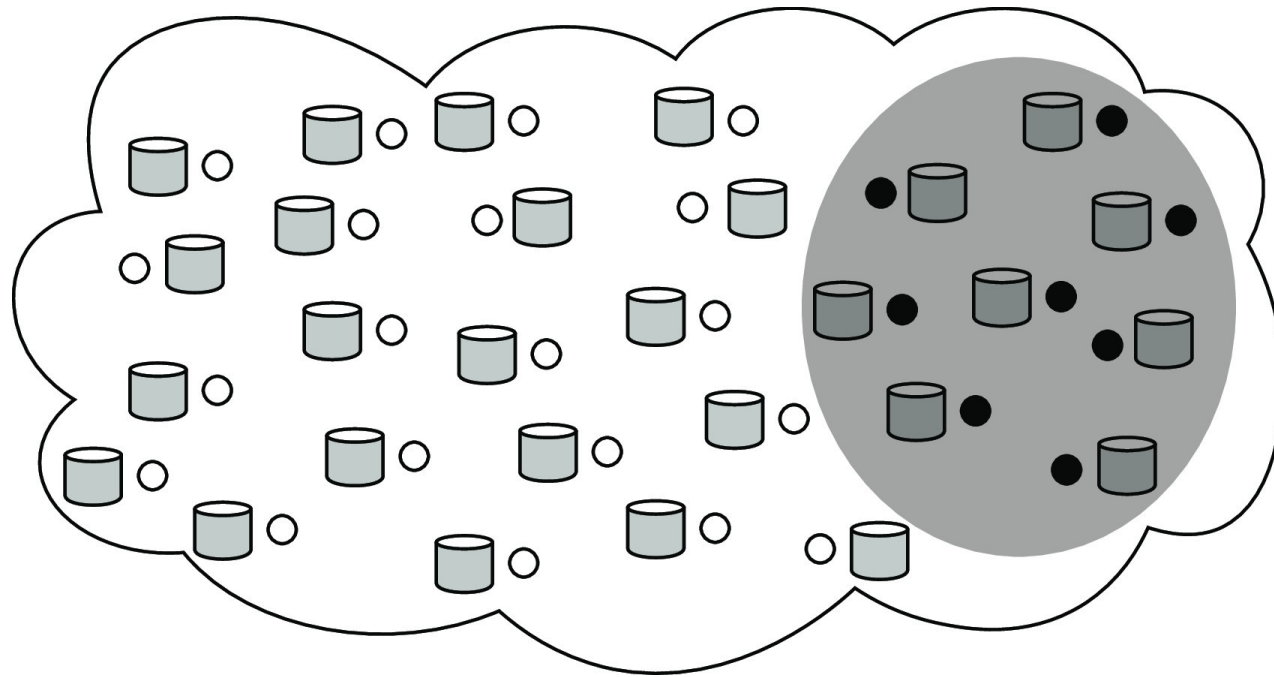


Observation

A single entity decides on which validator can go ahead and append a block. every node in a blockchain is supposed to have a full copy of the blockchain locally. A centralized blockchain architecture doesn't fit the design goals because the idea is to avoid having a single trusted third party.

Hybrid System: Blockchains

Distributed solution (permissioned)

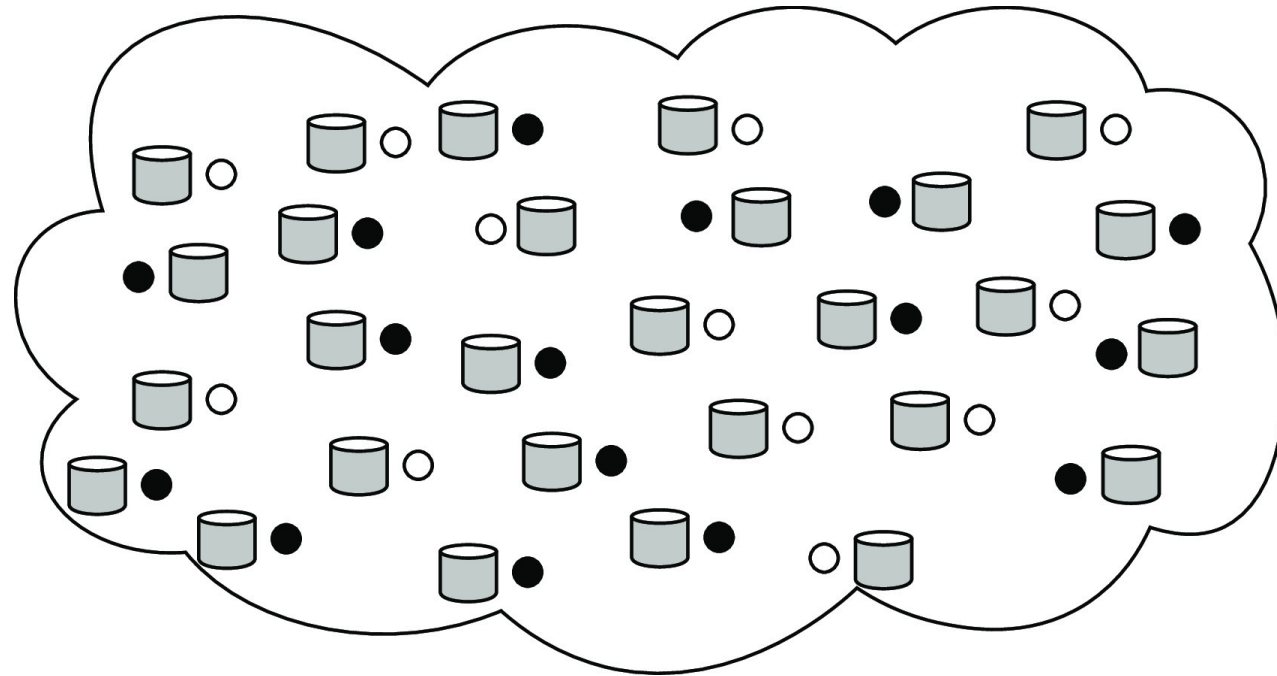


Observation

- A selected, relatively small group of servers jointly reach consensus on which validator can go ahead.
- None of these servers needs to be trusted, as long as roughly two-thirds behave according to their specifications.
- In practice, only a few tens of servers can be accommodated.

Hybrid System: Blockchains

Decentralized solution (permissionless)



Observation

- Participants collectively engage in a **leader election**. Only the elected leader is allowed to append a block of validated transactions.
- Large-scale, decentralized leader election that is fair, robust, secure, and so on, is far from trivial.

Thank You



MOVE FORWARD.
BE GREAT.